Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mcol functions

Synopsis

The functions prefixed with Mcol make up the Color Analysis module. The Color Analysis module allows you to perform color-based operations such as color transformation using relative color calibration, matching, projection, and distance. You can use these operations to quickly design powerful applications that can correct colors, identify colors, segment colors (supervised), detect defects, enhance color-to-grayscale conversion, separate superimposed colors, and measure the relative presence of a color in an image.

Functions

McolAlloc
McolAllocResult
McolControl
McolDefine
McolDistance
McolDraw
McolFree
McolGetResult
McolInquire
McolMask
McolMatch
McolPreprocess
McolProject
McolRestore
McolSave
McolSetMethod
McolStream
McolTransform

McolAlloc

Synopsis

Allocate a color matching context or a relative color calibration context.

Syntax

```
AIL_ID McolAlloc(
    AIL_ID SysId,               //in
    AIL_INT64 ContextType,      //in
    AIL_INT64 SourceColorSpace, //in
    AIL_ID ColorSpaceProfileId, //in
    AIL_INT64 ControlFlag,      //in
    AIL_ID *ContextIdPtr        //out
)
```

Description

The context contains all the information necessary to perform McolMatch() (for color matching) or McolTransform() (for relative color calibration), including global context settings and color-samples. To add color-samples to a context, use McolDefine(). To adjust context and color-sample settings, use McolControl() and McolSetMethod().

Aurora Imaging Library considers all color data to be the same as the context's color space, regardless of the color attributes of the data's buffer (MbufAllocColor()). For relative color calibration, color must be RGB (8-bit).

When the color matching context or relative color calibration context is no longer required, release it using McolFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Specifies the type of context to allocate. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COLOR_CALIBRATION_RELATIVE	Allocates a relative color calibration context for use with McolTransform().	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_MATCHING	Allocates a color matching context for use with McolMatch().	c		e	f	g	a		i		j	k	l	m	n	b

SourceColorSpace INQ

Specifies the source color space of the context. In general, all color data required for the color operation should be the same as the source color space. For example, if you specify an M_RGB source color space, color-samples (McolDefine()) and target or source images (McolMatch() or McolTransform()) should also typically be RGB.

This parameter should be set to one of the following values. Unless otherwise specified, values apply to both color matching and relative color calibration contexts.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the color space																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_RGB.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIELAB	Specifies a CIELAB (or LAB) color space. This is only for color matching contexts. M_CIELAB is based on the color's luminance (L), its position between red and green (A), and its position between yellow and blue (B). Store the components of CIELAB data in color buffers as follows: the L component in band 0, the A component in band 1, and the B component in band 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_HSL	Specifies an HSL color space. This is only for color matching contexts. M_HSL is based on the color's hue (H), saturation (S), and luminance (L). Store the components of HSL data in color buffers as follows: the H component in band 0, the S component in band 1, and the L component in band 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB	Specifies an RGB color space. M_RGB is based on the color's red (R), green (G), and blue (B) components. Store the components of RGB data in color buffers as follows: the R component in band 0, the G component in band 1, and the B component in band 2. For color matching, when setting the color operation strategy with McolSetMethod(), you can also specify that, for internal calculations, you want to convert your RGB data to CIELAB before the match operation (McolMatch()).	c		e	f	g	a		i		j	k	l	m	n	b

ColorSpaceProfileId

Specifies the context's color space profile. This is the standard Aurora Imaging Library uses to interpret the source color space data when performing an internal conversion between two distinct color spaces before the match operation (for example, specifying a CIELAB conversion mode with McolSetMethod() prior to calling McolMatch()). Unless you are performing such a conversion for the match, the color space profile standard has no effect.

This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the context's color space profile																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to use standard RGB specifications (sRGB), as defined by the International Electrotechnical Commission (IEC) Project Team 61966-2-1.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextIdPtr

Specifies the address of the variable in which to write the color matching context or relative calibration context identifier or specifies the data type that the function should use to return the color matching context or relative calibration context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated color matching context or relative color calibration context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated color matching context or relative color calibration context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_COL_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the color matching context or relative color calibration context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the calibration context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated relative color calibration context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the color matching context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated color matching context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the color matching context or relative calibration context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_COL_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolAllocResult

Synopsis

Allocate a color matching result buffer.

Syntax

```
AIL_ID McolAllocResult(
    AIL_ID SysId,          //in
    AIL_INT64 ResultType,  //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ResultIdPtr    //out
)
```

Description

This function allocates a result buffer, on the specified system, to store the results from an McolMatch() operation. You can read the results from the result buffer, using McolGetResult().

When the color matching result buffer is no longer required, release it using McolFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the color matching result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the result buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result buffer to allocate. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result buffer to allocate																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_COLOR_MATCHING_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

M_COLOR_MATCHING_RESULT	Specifies to allocate a result buffer for color matching results.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ResultIdPtr

Specifies the address of the variable in which to write the color matching result buffer identifier or specifies the data type that the function should use to return the color matching result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated color matching result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated color matching result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_COL_ID* is returned instead of a standard Aurora Imaging Library identifier. This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the color matching result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b

Address for the result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated color matching result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b	

Return value

The returned value is the color matching result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_COL_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolControl

Synopsis

Control a color matching context, its color-samples, or the color-samples of a relative color calibration context.

Syntax

```
void McolControl(
    AIL_ID ContextId,       //out
    AIL_INT IndexOrLabel,   //in
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function sets the specified control type for a color matching context, its color-samples, or the color-samples of a relative color calibration context. These control type settings affect the execution of McolPreprocess(), McolMatch(), McolTransform(), and McolDraw(). Typically, you can inquire the control type setting, using McolInquire().

Changing certain control type settings require you to preprocess the context again, using McolPreprocess(). To know if the context needs to be preprocessed, call McolInquire() with the M_PREPROCESSED inquire type.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **McolControl()** internally calls the *AIL_DOUBLE* version of this function (McolControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (McolControlInt64()).

Parameters

ContextId

Specifies the identifier of the color matching or relative color calibration context. The context must have been previously allocated on the required system using McolAlloc() with M_COLOR_MATCHING or M_COLOR_CALIBRATION_RELATIVE.

IndexOrLabel

Specifies what to control. Set this parameter to one of the following values. Unless otherwise specified, these values are only available for color matching contexts.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to control																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_INDEX(AIL_INT SampleIndex)	Specifies the index of the color-sample to control. This value is available for color matching and relative color calibration contexts.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

	SampleIndex Specifies an index value greater than or equal to 0.														
M_SAMPLE_LABEL(AIL_INT SampleLabel)	Specifies the label of the color-sample to control. This value is available for color matching and relative color calibration contexts.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleLabel Specifies a label value greater than 0.															
M_ALL	Specifies to control all color-samples within the color matching context. The specified control type setting must be supported by all the color-samples.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Specifies to control a global setting of the color matching context.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the setting to change.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For controlling general color matching context settings
- For controlling color matching context settings that are used for color space encoding
- For controlling color-samples

The following ControlType and corresponding ControlValue parameter settings are used to control general color matching context settings. In this case, you must set the IndexOrLabel parameter to M_CONTEXT. These values are only available for color matching contexts.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling general color matching context settings																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																

M_ACCEPTANCE	Sets the acceptance level for the color-sample's score (M_SCORE with McolGetResult()). This score indicates the similarity between the color of the color-sample and the color of the target area. The higher the acceptance, the closer the colors must be for them to match. For a match, the color-sample must have a score that is greater than or equal to this level. INQ														
M_DEFAULT	Specifies the default value; the default value is 0.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCEPTANCE_RELEVANCE	Sets the acceptance level for the target area's relevance score (M_SCORE_RELEVANCE). This score indicates the significance (relevance) of the match score (M_SCORE). In statistics, this is similar to the confidence level; that is, it represents how confident you can be that the best-matched color-sample actually is the best possible match. For example, a high relevance acceptance indicates that the best-matched color-sample must be vastly superior to all other color-samples, while a low relevance acceptance indicates that the best-matched color-sample only needs to be marginally superior to the other color-samples. A target area can only have a successful matching status (M_STATUS with McolGetResult()) if its relevance score is above the relevance acceptance level. A target area with a successful matching status also requires at least one color-sample score to be above M_ACCEPTANCE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable relevance score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

M_BACKGROUND_DRAW_COLOR	Sets the color to use to draw background pixels when drawing or generating the color images (M_DRAW_..._USING_COLOR), the distance image (M_DRAW_DISTANCE), or the grayscale label images (M_DRAW_..._USING_LABEL), using McolDraw() or McolMatch(). A pixel is considered background when it is outside the target areas. For background pixels to exist, you must specify an area identifier image when calling McolMatch(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_TRANSPARENT.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR888(AIL_INT ComponentBand0, AIL_INT ComponentBand1, AIL_INT ComponentBand2)	Specifies a 24-bit color value (eight bits per band) with which to set the background pixels. The color value is taken as is (no conversion), according to the source color space. Typically, this should be RGB, since the background color is meant for display. When M_BACKGROUND_DRAW_COLOR is set with M_COLOR888(), only the first component (for example, red) will be used to draw or generate M_DRAW_DISTANCE, which is a grayscale image.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	ComponentBand0 Specifies the first component, as a value between 0 and 255.														
	ComponentBand1 Specifies the second component, as a value between 0 and 255.														
	ComponentBand2 Specifies the third component, as a value between 0 and 255.														
M_TRANSPARENT	Specifies not to change the background. Therefore, the background pixels will be those of the specified destination image.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies a grayscale value for the background pixels, as a *double* or *integer*. When drawing the distance image (M_DRAW_DISTANCE) in a 32-bit integer buffer, you should keep the M_BACKGROUND_DRAW_COLOR value under 24-bit (that is, less than 16,777,215).	c		e	f	g	a		i		j	k	l	m	n	b

M_BAND_MODE	Specifies the color bands to use when performing a match. This applies to all color-samples in the context and to the target areas. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL_BANDS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_BANDS	Specifies to use all bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BAND_0	Specifies to use band 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BAND_0 + M_COLOR_BAND_1	Specifies to use bands 0 and 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BAND_0 + M_COLOR_BAND_2	Specifies to use bands 0 and 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BAND_1	Specifies to use band 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BAND_1 + M_COLOR_BAND_2	Specifies to use bands 1 and 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BAND_2	Specifies to use band 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVERSION_GAMMA	Specifies whether to remove gamma correction when internally converting RGB source colors before the match using McolSetMethod() with the CIELAB conversion mode. Gamma correction refers to color data that has been processed, typically during data acquisition (that is, by the camera doing the grab), to compensate for a non-linear transformation between a pixel's value and its displayed intensity on the display device. When converting RGB source colors to CIELAB, Aurora Imaging Library requires linear color data; that is, data that is uncorrected. Therefore, if gamma correction has been applied on the RGB source color data, you must remove it. Unless you are using the CIELAB conversion mode in an RGB source color space, M_CONVERSION_GAMMA is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies not to remove gamma correction. In this case, your RGB color data should not be in a corrected state (it should be linear). For example, the camera capturing the RGB color data has not applied a gamma correction.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to remove gamma correction. In this case, your RGB color data should be in a corrected state (it should be non-linear). For example, the camera capturing the RGB color data has applied a gamma correction. Aurora Imaging Library assumes that your color data has been corrected according to the standards of the context's reference color space, as specified using McolAlloc() with the ColorSpaceProfileId parameter. Aurora Imaging Library uses these standards (for example, sRGB) to remove the gamma correction.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_IMAGE_NORMALIZE		Sets the normalization factor when drawing or generating M_DRAW_DISTANCE, using McolDraw() or McolMatch(). Distance results are always clipped at the distance buffer's maximal possible value, which can occur when specifying M_NO_NORMALIZE or a specific normalization factor. For example, when using an 8-bit unsigned distance buffer, distances higher than 255 will be written as 255. Note that if you specify a distance buffer of type *floating-point*, distance results would almost never be greater than the buffer's maximal possible value, since its limit is extremely high. Aurora Imaging Library ignores M_DISTANCE_IMAGE_NORMALIZE when using McolSetMethod() with M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NO_NORMALIZE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_MAX_NORMALIZE	Specifies to normalize distance values according to the greatest calculated distance. This value corresponds to the result M_MAX_DISTANCE (McolGetResult()). The normalization performed is: • (*Distance/GreatestCalculatedDistance*) x *MaximumPossibleValueOfDistBuf* (for distance buffer's of type *integer*). • (*Distance/GreatestCalculatedDistance*) (for distance buffer's of type *floating-point*). Note that if the distance buffer is of type *floating-point*, you would typically not normalize the distance results, since all this will do is return a value between 0.0 and 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_NORMALIZE	Specifies no normalization. For distance buffers of type *integer*, distance values are truncated (no decimals). For distance buffers of type *floating-point*, distance values are complete (with decimals).	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies to normalize distance values according to the specified normalization value. The normalization performed is: • (*Distance/NormalizationValue*) x *MaximumPossibleValueOfDistBuf* (for distance buffer's of type *integer*). • (*Distance/NormalizationValue*) (for distance buffer's of type *floating-point*). Note that if the distance buffer is of type *floating-point*, you would typically not normalize the distance results, since all this will do is return a value between 0.0 and 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TOLERANCE_MODE		Sets the strategy with which to use the tolerance value (M_DISTANCE_TOLERANCE) to calculate the acceptable (matching) color distance (tolerance) between the color-sample and a target area. When using an M_HISTOGRAM_MATCHING operation mode (McolSetMethod()), you must set M_DISTANCE_TOLERANCE_MODE to M_ABSOLUTE. When using an M_HISTOGRAM_VOTE operation mode, Aurora Imaging Library ignores M_DISTANCE_TOLERANCE_MODE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ABSOLUTE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ABSOLUTE	Specifies that the M_DISTANCE_TOLERANCE value will be untouched and used as the acceptable distance tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RELATIVE	Specifies a tolerance strategy based on the relative distances between pairs of color-samples. In this case, Aurora Imaging Library establishes the smallest distance between any given color-sample and every other color-sample. Half this value is then multiplied by the M_DISTANCE_TOLERANCE value, and the resulting value serves as the acceptable distance tolerance for the color-sample in question. This process is repeated for each color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAMPLE_STDDEV	Specifies a tolerance strategy based on the color-sample's standard deviation. For color-samples that come from an image (McolDefine() with M_IMAGE), the color-sample's standard deviation is computed as the distance between each pixel in the color-sample and the average color of the color-sample. Aurora Imaging Library calculates this distance according to the current distance type set with McolSetMethod(). The resulting distance, which in this case is considered to be the color-sample's standard deviation, is multiplied by M_DISTANCE_TOLERANCE and is used as the acceptable distance tolerance. If the computed color-sample's standard deviation is less than 1.0, Aurora Imaging Library uses 1.0 as the standard deviation. For triplet color-samples (McolDefine() with M_TRIPLET), Aurora Imaging Library uses 1.0 as the standard deviation. In this case, M_DISTANCE_TOLERANCE remains unchanged and is used as the acceptable distance tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERATE_DISTANCE_IMAGE		Sets whether to generate the distance image and store it in the result buffer. This image contains the difference between the color of the target area and the color of its best-matched color-sample. Aurora Imaging Library ignores M_GENERATE_DISTANCE_IMAGE when using McolSetMethod() with M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies not to generate the image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to generate the image.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERATE_PIXEL_MATCH		Sets whether to generate the image containing the label of the color-sample for which each pixel voted. This image is only available if you use McolSetMethod() with M_MIN_DIST_VOTE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to generate the image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to generate the image.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERATE_SAMPLE_COLOR_LUT		Sets whether to generate the color-sample label LUT and store it in the result buffer. This LUT contains the label value of each color-sample and its associated average color. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to generate the LUT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to generate the LUT.	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_BINS_BAND_0		Sets the number of histogram bins for color band 0. Relevant only when matching colors using McolSetMethod() with M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE. The number of bins situated along the axis of band 0 determine the size of the bins for that band. If you multiply the number of bins situated along the axis of each band (M_NB_BINS_BAND_0, M_NB_BINS_BAND_1, and M_NB_BINS_BAND_2) you obtain the total number of bins. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of histogram bins for color band 0.	c		e	f	g	a		i		j	k	l	m	n	b

M_NB_BINS_BAND_1	Sets the number of histogram bins for color band 1. Relevant only when matching colors using McolSetMethod() with M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE. The number of bins situated along the axis of band 1 determine the size of the bins for that band. If you multiply the number of bins situated along the axis of each band (M_NB_BINS_BAND_0, M_NB_BINS_BAND_1, and M_NB_BINS_BAND_2) you obtain the total number of bins. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Number of histogram bins for color band 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_BINS_BAND_2	Sets the number of histogram bins for color band 2. Relevant only when matching colors using McolSetMethod() with M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE. The number of bins situated along the axis of band 2 determine the size of the bins for that band. If you multiply the number of bins situated along the axis of each band (M_NB_BINS_BAND_0, M_NB_BINS_BAND_1, and M_NB_BINS_BAND_2) you obtain the total number of bins. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Number of histogram bins for color band 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIER_DRAW_COLOR	Sets the color to use to draw outlier pixels when drawing or generating the color images (M_DRAW_..._USING_COLOR) or the distance image (M_DRAW_DISTANCE), using McolDraw() or McolMatch(). Outlier pixels are pixels inside a target area that do not match with any color-sample. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR888(AIL_INT ComponentBand0, AIL_INT ComponentBand1, AIL_INT ComponentBand2)	Specifies a 24-bit color value (eight bits per band) with which to set the outlier pixels. The color value is taken as is (no conversion), according to the source color space. Typically, this should be RGB, since the outlier color is meant for display. When M_OUTLIER_DRAW_COLOR is set with M_COLOR888(), only the first component (for example, red) will be used to draw or generate M_DRAW_DISTANCE, which is a grayscale image.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters														
		ComponentBand0 Specifies the first component, as a value between 0 and 255.														
		ComponentBand1 Specifies the second component, as a value between 0 and 255.														
		ComponentBand2 Specifies the third component, as a value between 0 and 255.														
	M_TRANSPARENT	Specifies not to change the value of the outlier pixels. Therefore, the outlier pixels will be those of the specified image.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies a grayscale value for the outlier pixels, as a *double* or *integer*. When drawing the distance image (M_DRAW_DISTANCE) in a 32-bit *integer* buffer, you should keep the M_OUTLIER_DRAW_COLOR value under 24-bit (that is, less than 16,777,215).	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIER_LABEL		Sets the label to use for outlier pixels when drawing or generating the label images (M_DRAW_..._USING_LABEL) using McolDraw() or McolMatch(). Outlier pixels are pixels inside a target area that do not match with any color-sample. Note that the M_OUTLIER_LABEL setting does not represent a valid color-sample and you cannot pass it as a color-sample label to McolInquire(), McolGetResult(), or McolDraw(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0	Specifies the label of the outlier pixels, as an *integer*. The label must be unique among existing labels, and must be less than 2^{24} .	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_AREA_IMAGE		Sets whether to save the area identifier image in the result buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to save the image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to save the image.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control color space encoding, according to the actual dynamic range of your color data. In this case, you must set the IndexOrLabel parameter to M_CONTEXT. These values are only available for color matching contexts.

Color space encoding defines how color is transformed from the range represented in an image buffer to its native (theoretical) range. For example, RGB color space data is represented in an image buffer as values between 0 and 255 (8-bit); these values are then mapped to their native data range, which consists of all real numbers between 0 and 1.

The Color Analysis module allows you to perform this transformation automatically, using M_ENCODING with M_nBIT, which is typically sufficient. You can also use M_USER_DEFINED to explicitly control the transformation according to offset (M_OFFSET_BAND_n) and scale (M_SCALE_BAND_n) values. For more information, see the Color space encoding subsection of the Advanced color matching settings and concepts section of Chapter 22: Color processing and analysis.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling color matching context settings that are used for color space encoding																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																	
M_ENCODING	Sets the color space encoding, which defines how color is transformed from the range represented in an image buffer to its native (theoretical) range. INQ																
M_DEFAULT	Same as M_8BIT.	c		e	f	g	a		i		j	k	l	m	n	b
M_8BIT	Specifies an internally defined color space encoding for 8-bit images.	c		e	f	g	a		i		j	k	l	m	n	b
M_10BIT	Specifies an internally defined color space encoding for 10-bit images.	c		e	f	g	a		i		j	k	l	m	n	b

	M_12BIT	Specifies an internally defined color space encoding for 12-bit images.	c		e	f	g	a		i		j	k	l	m	n	b
	M_14BIT	Specifies an internally defined color space encoding for 14-bit images.	c		e	f	g	a		i		j	k	l	m	n	b
	M_16BIT	Specifies an internally defined color space encoding for 16-bit images.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies that the color space encoding is defined using M_OFFSET_BAND_n and M_SCALE_BAND_n.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_BAND_0		Sets the offset for the color space encoding, for band 0. This only applies if M_ENCODING is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the offset, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_BAND_1		Sets the offset for the color space encoding, for band 1. This only applies if M_ENCODING is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the offset, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_BAND_2		Sets the offset for the color space encoding, for band 2. This only applies if M_ENCODING is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the offset, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCALE_BAND_0		Sets the scale for the color space encoding, for band 0. This only applies if M_ENCODING is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the scale, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCALE_BAND_1		Sets the scale for the color space encoding, for band 1. This only applies if M_ENCODING is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the scale, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b

M_SCALE_BAND_2		Sets the scale for the color space encoding, for band 2. This only applies if M_ENCODING is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the scale, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control color-samples defined in the context. In this case, you must set the IndexOrLabel parameter to M_SAMPLE_INDEX() or M_SAMPLE_LABEL(). Unless otherwise specified, these values are only available for color matching contexts.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling color-samples																		
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_DISTANCE_TOLERANCE		Sets the acceptable tolerance for the color distance between the color-sample and a target area. The tolerance ranges from no color difference to M_DISTANCE_TOLERANCE. The greater the tolerance, the greater the distance (difference) between the colors can be, for them to match. Aurora Imaging Library ignores M_DISTANCE_TOLERANCE when using McolSetMethod() with M_HISTOGRAM_VOTE. INQ															
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies the tolerance automatically. If you set M_DISTANCE_TOLERANCE_MODE to M_ABSOLUTE, M_AUTO specifies an infinite (M_INFINITE) tolerance. If you set M_DISTANCE_TOLERANCE_MODE to M_RELATIVE, M_AUTO specifies a tolerance of 1. If you set M_DISTANCE_TOLERANCE_MODE to M_SAMPLE_STDDEV, M_AUTO specifies a tolerance of 3.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies an infinite tolerance.	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0.0	Specifies the tolerance value. Set the tolerance according to the specified tolerance mode (M_DISTANCE_TOLERANCE_MODE) and distance type (McolSetMethod()). For example, a tolerance value of 1 when using an M_MAHALANOBIS distance type is not the same as when using an M_MANHATTAN distance type, even if the tolerance mode is M_ABSOLUTE.	c		e	f	g	a		i		j	k	l	m	n	b
M_HISTOGRAM_FREQUENCY_THRESHOLD		Sets the minimal frequency, in pixels, for a histogram bin to count in the matching process. This setting is only valid when using McolSetMethod() with M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE. Increasing the minimal frequency can lead to a faster match while discarding noise in the sample histogram. Bins with a frequency less than the threshold are considered empty. When using triplet color-samples with M_HISTOGRAM_VOTE, it is not recommended to set the frequency threshold to a value other than 1, which is the default. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1 pixel.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the minimal frequency, as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LABEL_VALUE		Sets the color-sample's label. This value is available for color matching and relative color calibration contexts. Initially, you specify a color-sample's label value when adding a color-sample to the context, using McolDefine(). You can use M_SAMPLE_LABEL_VALUE to change the label afterward. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label, as an *integer*. The label must be unique among existing labels, and must be less than 2^{24} .	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void McolControlInt64 (AIL_ID ContextId, AIL_INT IndexOrLabel, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextId

See ContextId of the main function for a description.

IndexOrLabel

See IndexOrLabel of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void McolControlDouble (AIL_ID ContextId, AIL_INT IndexOrLabel, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextId

See ContextId of the main function for a description.

IndexOrLabel

See IndexOrLabel of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolDefine

Synopsis

Define or delete a color-sample or color element.

Syntax

```
void McolDefine(
    AIL_ID ContextId,          //in
    AIL_ID SrcImageBufId,      //in
    AIL_INT UserLabelOrIndex,  //in
    AIL_INT64 ColorSampleType, //in
    AIL_DOUBLE Param1,         //in
    AIL_DOUBLE Param2,         //in
    AIL_DOUBLE Param3,         //in
    AIL_DOUBLE Param4          //in
)
```

Description

This function allows you to add a color-sample to, or remove a color-sample from, a color matching or relative color calibration context. You can also use this function to add color elements to, or remove elements from, an existing color-sample. When defining a color-sample, you are, by default, defining the first color element of the color-sample. To define a color-sample (and consequently the first color element), you can use an image or three explicit color band values.

For a relative color calibration context, you must use this function to define or delete the reference color-sample. Information about color-samples applies to the reference color-sample unless otherwise specified.

A color matching or relative color calibration context must have one or more color-samples. A relative color calibration context must have one reference color-sample.

Aurora Imaging Library uses color-samples to execute McolMatch() (color matching) and McolTransform() (relative color calibration). Before calling these functions, you must preprocess the context that holds the color-samples, using McolPreprocess(). Whenever you add, add to, delete, or delete from a color-sample, you must preprocess the context for these changes to take effect. You must add, or add to, color-samples one at a time; however, you can delete all color-samples at once.

When defining image type color-samples for a color matching context, Aurora Imaging Library performs a color estimation, which is based on color statistics, such as the mean. The estimate is considered the color of the color-sample and is used in subsequent operations, such as McolMatch() and McolDraw(). When defining image type color-samples for a relative color calibration context, Aurora Imaging Library does not typically perform a color estimation. Aurora Imaging Library considers the color of the color-sample to be the color data of each individual pixel. This is what Aurora Imaging Library uses in subsequent operations that require color-samples, such as McolTransform() and McolDraw(). In some cases, you can use the mean color of the color-sample instead (McolSetMethod()).

The size of color-samples does not usually affect the operations that use them.

Color-sample indices start at 0. Each subsequent color-sample you add to the context is given a sequential index number. If you delete a color-sample, Aurora Imaging Library shifts all entries with higher indices down by one. This convention also applies when adding or deleting a color element to or from a color-sample. The reference color-sample does not have an index (there can only be one).

You can also assign a label to a color-sample. If you don't, Aurora Imaging Library assigns one automatically. To

change it afterward, use McolControl() with M_SAMPLE_LABEL_VALUE. Adding color elements to, or removing color elements from, a color-sample does not change its index or label. You cannot assign a label to the reference color-sample (there can be only one, to which Aurora Imaging Library automatically assigns a label).

You can limit the McolMatch() or McolTransform() operation to a region of the image buffer, using a rectangular region of interest (ROI) set using MbufSetRegion().

Aurora Imaging Library assumes all the color you are using is in the context's source color space (McolAlloc()). Therefore all color elements that you add must be consistent. For example, if you allocate an RGB context, bands 0, 1, and 2 are assumed to be red, green, and blue; you will therefore not receive an error if you try to match RGB target areas with HSL color-samples. Similarly, you will not receive an error if you try to match a 16-bit target area with an 8-bit color-sample. Even when the color-samples are inconsistent, Aurora Imaging Library still processes them as is, which can produce misleading results. However, Aurora Imaging Library will not allow you to add color elements of different types to the same color-sample. If you do, you will receive an error. For relative color calibration, all color data must be RGB. Aurora Imaging Library processes it as RGB even if it is not.

Color space encoding must be set according to the actual dynamic range of your color elements. By default, Aurora Imaging Library assumes an 8-bit color space encoding (regardless of the buffer's depth). For color matching, you can specify a different encoding, such as 16-bit, using McolControl() with M_ENCODING. For relative color calibration, Aurora Imaging Library always assumes 8-bit. No other encoding can be set.

You can use McolMask() to mask regions of a color element in a specified color-sample.

Parameters

ContextId

Specifies the identifier of the context in which to add or delete a color-sample. The context must have been previously allocated on the required system using McolAlloc() with M_COLOR_MATCHING or M_COLOR_CALIBRATION_RELATIVE. If you are adding or deleting a color element from a color-sample, this parameter specifies the context in which the color-sample you wish to add or delete elements from is located.

SrcImageBufId

Specifies the identifier of the source image buffer to use to define or add elements to a color-sample. If you are defining or adding elements to a color-sample from three explicit color band values, or if you are deleting color-samples or color elements, set this parameter to **M_NULL**.

This image buffer can have a region of interest (ROI) defined using MbufSetRegion(). The region must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in vector format (M_VECTOR).

If the source image buffer has an ROI, Aurora Imaging Library ignores all pixels outside the ROI when defining an M_IMAGE color-sample. In this case, you must set Param1 to Param4 to **M_DEFAULT**.

UserLabelOrIndex

Specifies a color-sample. You can add the color-sample to, or delete the color-sample (or all color-samples) from, the specified context (ContextId). You can also add color elements to, or delete color elements from, the color-sample.

Set this parameter to one of the following values. Unless otherwise specified, values apply to both color matching and relative color calibration contexts.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a color-sample

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Adds a color-sample to the context and automatically assigns it a label. The label is the next available numerical value following the highest label defined.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFERENCE_SAMPLE_ITEM(AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to the reference color-sample. Use M_REFERENCE_SAMPLE_ITEM() to delete a color element from the reference color-sample. M_REFERENCE_SAMPLE_ITEM() only applies to relative color calibration contexts.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ElementIndex Specifies the index of the color element (item). The index value must be greater than or equal to 0.															
M_SAMPLE_INDEX(AIL_INT SampleIndex)	Specifies the index of an existing color-sample. Use M_SAMPLE_INDEX() to delete a color-sample from the context, or to add a color element to a color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleIndex Specifies an index value greater than or equal to 0.															
M_SAMPLE_INDEX_ITEM(AIL_INT SampleIndex, AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to a color-sample index. Use M_SAMPLE_INDEX_ITEM() to delete a color element from a color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleIndex Specifies the index of the color-sample that holds the color element (item). The index must be greater than or equal to 0.															
	ElementIndex Specifies the index of the color element (item). The index value must be greater than or equal to 0.															

M_SAMPLE_LABEL(AIL_INT SampleLabel)	Specifies the label of a new or an existing color-sample. Use M_SAMPLE_LABEL() to: add a new color-sample to the context, add a color element to an existing color-sample, or delete an existing color-sample from the context.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleLabel INQ Specifies a label value greater than 0. The label cannot use the last valid label value (2^{12-1}). It is reserved for the reference color-sample (M_REFERENCE_SAMPLE).															
M_SAMPLE_LABEL_ITEM(AIL_INT SampleLabel, AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to a color-sample label. Use M_SAMPLE_LABEL_ITEM() to delete a color element from a color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleLabel Specifies the label of the color-sample that holds the color element (item). The label value must be greater than 0.															
	ElementIndex Specifies the index of the color element (item). The index value must be greater than or equal to 0.															
M_ALL	Specifies all color-samples. Use M_ALL to delete all color-samples from the context. For relative color calibration contexts, M_ALL also deletes the reference color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFERENCE_SAMPLE	Specifies the reference color-sample. Use M_REFERENCE_SAMPLE to: add the reference color-sample to the context, delete the reference color-sample from the context, or add a color element to the reference color-sample. Aurora Imaging Library automatically assigns the last possible label value (2^{12-1}) to M_REFERENCE_SAMPLE. M_REFERENCE_SAMPLE only applies to relative color calibration contexts.	c		e	f	g	a		i		j	k	l	m	n	b

ColorSampleType INQ

Specifies the type of the color-sample or color element.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies an attribute of the color-sample or color element. Its definition is dependent on the color-sample or color element type chosen.

Set this parameter to **M_DEFAULT** if not used, or if the source image buffer has a region of interest (ROI), that was set using MbufSetRegion().

See the Parameter associations section for possible values that can be specified.

Param2

Specifies an attribute of the color-sample or color element. Its definition is dependent on the color-sample or color element type chosen.

Set this parameter to **M_DEFAULT** if not used, or if the source image buffer has a region of interest (ROI), that was set using MbufSetRegion().

See the Parameter associations section for possible values that can be specified.

Param3

Specifies an attribute of the color-sample or color element. Its definition is dependent on the color-sample or color element type chosen.

Set this parameter to **M_DEFAULT** if not used, or if the source image buffer has a region of interest (ROI), that was set using MbufSetRegion().

See the Parameter associations section for possible values that can be specified.

Param4

Specifies an attribute of the color-sample or color element. Its definition is dependent on the color-sample or color element type chosen.

Set this parameter to **M_DEFAULT** if not used, or if the source image buffer has a region of interest (ROI), that was set using MbufSetRegion().

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ColorSampleType**, **Param1**, **Param2**, **Param3**, and **Param4** parameters.

- For adding a color-sample or color element
- For deleting color-samples from the context, or color elements from the specified color-sample

Set all unused parameters to **M_DEFAULT**.

The following ColorSampleType parameter settings can be used to add a color-sample to the context, or a color element to the specified color-sample.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For adding a color-sample or color element																
ColorSampleType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

Param1																
Param2																
Param3																
Param4																
M_IMAGE +	Specifies to add a color-sample or color element using the specified region of the source image. In this case, you must set the SrcImageBufId parameter to the identifier of an 8- or 16-bit color (3-band) unsigned image buffer, if you are using a color matching context. For a relative color calibration context, the buffer must be an 8-bit RGB color (3-band) image buffer.															
Param1	Sets the X-offset of the origin of the color-sample or color element region from the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that the X-offset is equal to 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the Y-offset of the origin of the color-sample or color element region from the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that the Y-offset is equal to 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the width (X-size) of the color-sample or color element region.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that the width is from the origin of the color-sample or color element region to the end of the source image, in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Param4	Sets the height (Y-size) of the color-sample or color element region.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that the height is from the origin of the color-sample or color element region to the end of the source image, in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRIPLET +		Specifies to add a color-sample or color element using three explicit values for the color bands (RGB, HSL, or CIELAB). In this case, you must set the SrcImageBufId parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Sets the value of the first color band. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Param2	Sets the value of the second color band. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Param3	Sets the value of the third color band. INQ	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For adding a color-sample or color element.

You can add the following value to the above-mentioned values to specify additional color elements to add to an already existing color-sample.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For adding elements to an already existing color-sample																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ADD_COLOR_TO_SAMPLE	Adds the specified color element to the specified color-sample. If the specified color-sample does not already exist, you will get an error. The color estimation that Aurora Imaging Library performs for the color-sample uses the new elements as well as the existing element(s). Aurora Imaging Library considers the resulting estimate the color of the color-sample and uses it in subsequent operations, such as color matching, relative color calibration, and drawing. All color elements in a color-sample must have the same type. For example, if the existing color-sample type is M_IMAGE, its additional color element(s) must also be M_IMAGE. Similarly, if the color-sample is M_TRIPLET, the additional element(s) must be M_TRIPLET. For M_IMAGE, you must ensure that the source image from which to define the additional color element(s) has the same format as the original source image used to define the existing color-sample.	c		e	f	g	a		i		j	k	l	m	n	b

The following ColorSampleType parameter setting can be used to remove color-samples from the context, or color elements from the specified color-sample. In this case, you must set the SrcImageBufId parameter to **M_NULL**, and Param1 to Param4 to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For deleting color-samples from the context, or color elements from the specified color-sample																	
ColorSampleType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	Param1																

	Param2	
	Param3	
	Param4	
M_DELETE		Specifies to delete one or all color-samples or color elements, depending on the UserLabelOrIndex parameter setting. Note that deleting the last remaining color-sample element (item) from a color-sample is equivalent to deleting the color-sample itself.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolDistance

Synopsis

Calculate the distance between colors.

Syntax

```
void McolDistance(
    AIL_ID Src1Id,                    //in
    AIL_ID Src2Id,                    //in
    AIL_ID DestId,                    //out
    AIL_ID MaskId,                    //in
    AIL_ID DistanceAttributesArrayId, //in
    AIL_INT64 DistType,               //in
    AIL_DOUBLE NormalizeValue,        //in
    AIL_INT64 ControlFlag             //in
)
```

Description

This function calculates the point-to-point distance between colors in two images. You can also calculate the point-to-point distance between the color in one image and: a color constant, a covariance matrix, or the covariance of a specified image. The calculated distance (for each pixel) is written in the specified destination image (DestId parameter). To ignore unwanted pixels in the distance calculation, you can apply a mask.

The color distance is typically calculated only for pixels within the intersection of the source (Src1Id and Src2Id), destination (DestId), and mask (MaskId) images, with the assumption of a common origin at the top-left corner.

The Color Analysis module assumes that the color of the images you use with **McolDistance**() is consistent. For example, you will not receive an error if you try to calculate the distance between an RGB and an HSL image, or a 16-bit and an 8-bit image. Even when color data is inconsistent, it is still processed as is, which can produce misleading results.

You can also obtain distance results after calling McolMatch(), which unlike **McolDistance**(), considers the color matching context's color space. For more information on how distances differ between these two functions, see the Distance between colors section of Chapter 22: Color processing and analysis.

> This function is optimized for planar buffers or for packed 32-bit BGR buffers with a width (X-size) greater than seven pixels (MbufAllocColor()). It is recommended to avoid providing both planar and packed buffers, since this requires some internal compensation.

Parameters

Src1Id

Specifies the identifier of the first source image buffer. The buffer must be an 8- or 16-bit color (3-band) unsigned image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Src2Id

Specifies the identifier of the second source buffer, which can be either an image or an Aurora Imaging Library array buffer.

For an image, the buffer must be an 8- or 16-bit color (3-band) unsigned image buffer. For an array, the content and size depends on the type of distance operation chosen (DistType).

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Note that when setting this parameter to an array buffer, this function will achieve maximal speed performance if the buffer's data type is the same as the data type of the first source image. For example, you will achieve maximal speed if the first source image buffer and second source array buffer are both 8-bit unsigned buffers.

DestId

Specifies the identifier of the destination image buffer in which to write the results of the distance calculation. The destination buffer must be a 1-band buffer; its type is not restricted.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Since distance results are numerical values containing decimals, if you specify the destination buffer as type *integer*, the fractional part of the value will be truncated. For example, a distance of 50.25 will truncate to 50.

Distance results are clipped if they are greater than the destination buffer's maximal possible value. For example, when using an 8-bit unsigned destination buffer, distances higher than 255 will be written as 255. Note that if you specify a destination buffer of type *floating-point*, distance results would almost never be greater than the buffer's maximal possible value, since its limit is extremely high.

To normalize distance results, use the NormalizeValue parameter.

MaskId

Specifies the identifier of the image buffer used to identify the masked (non-zero) pixels. This must be a 1- or 8-bit (1-band) unsigned buffer. To not apply a mask, set this parameter to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

The color distance is calculated only for masked (non-zero) pixels within the intersection of the source, destination, and mask images, with the assumption of a common origin at the top-left corner. Set unmasked pixels to 0; unmasked pixels are ignored in the distance calculation.

DistanceAttributesArrayId

Reserved for future expansion and must be set to **M_DEFAULT**.

DistType

Specifies the type of distance operation to perform. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the distance type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DELTA_E	Specifies to use a Delta-E color distance, as defined by the International Commission on Illumination (CIE). A Delta-E color distance is the same as a Euclidean color distance (M_EUCLIDEAN), but specialized for the LAB (CIELAB) color space. This function assumes that the data is in the LAB color space; it does not convert the data before computing the distance. To convert the data, use MimConvert() with M_SRGB_LINEAR_TO_LAB or M_SRGB_TO_LAB.	c		e	f	g	a		i		j	k	l	m	n	b
M_EUCLIDEAN	Specifies to use a Euclidean color distance. A Euclidean distance is the square root of the sum of the squared differences between the color of each pixel in the first source image buffer (Src1Id) and either the color of each pixel in the second source image buffer, or a color constant (Src2Id). For a color constant, you must use a 3x1, one-dimensional Aurora Imaging Library array buffer. This buffer's type is unrestricted, and must contain three color band values (for example, RGB).	c		e	f	g	a		i		j	k	l	m	n	b
M_MAHALANOBIS	Specifies to use a Mahalanobis color distance. A Mahalanobis distance is calculated between the color of each pixel in the image specified in the first source image buffer (Src1Id) and, typically, either the covariance of the image specified in the second source image buffer, or a covariance matrix (Src2Id). This distance, between a color and a distribution of colors, is similar to a Euclidean distance between the mean of the two colors, but weighted by the inverse of the covariance of the distribution. This implies that the more a color distribution varies in a direction within the color space, the less important is the distance in that direction. Since the covariance matrix of the second source is used, the second source should usually be a distribution of colors, such as an image, and not a single solid color (a color constant). However, if you provide a color constant as the second source, Mahalanobis behaves very much like Euclidean and will yield similar results. For a covariance matrix, you must use a 4x3, two-dimensional Aurora Imaging Library array buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_MANHATTAN	Specifies to use a Manhattan color distance. A Manhattan distance is the sum of the absolute value of the differences between the color of each pixel in the image specified in the first source image buffer (Src1Id) and either the color of each pixel in the second source image buffer, or a color constant (Src2Id). Note that in HSL the color's hue is stored as a separate band (H) represented as an angular position on a circular color disk. Therefore the distance between colors is equal to the smallest angular difference, rather than the absolute value of the difference. For a color constant, you must use a 3x1, one-dimensional Aurora Imaging Library array buffer. The buffer's type is unrestricted, and must contain three color band values (for example, RGB).	c		e	f	g	a		i		j	k	l	m	n	b

NormalizeValue

Specifies the factor with which to normalize the resulting distances. The normalized distance that is written to the destination buffer must respect this buffer's limits (DestId).

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the normalization of distances																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_NO_NORMALIZE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_NORMALIZE	Specifies to normalize distance values according to the greatest calculated distance. This value corresponds to the result M_MAX_DISTANCE (McolGetResult()). The normalization performed is: • (*Distance/GreatestCalculatedDistance*) x *MaximumPossibleValueOfDestBuf* (for destination buffer's of type *integer*). • (*Distance/GreatestCalculatedDistance*) (for destination buffer's of type *floating-point*). Note that if the destination buffer is of type *floating-point*, you would typically not normalize the distance results, since all this will do is return a value between 0.0 and 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_NORMALIZE	Specifies no normalization. For destination buffers of type *integer*, distance values are truncated (no decimals). For destination buffers of type *floating-point*, distance values are complete (with decimals).	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0.0	Specifies to normalize distance values according to the specified normalization value. The normalization performed is: • (*Distance*/*NormalizationValue*) x *MaximumPossibleValueOfDestBuf* (for destination buffer's of type *integer*). • (*Distance*/*NormalizationValue*) (for destination buffer's of type *floating-point*). Note that if the destination buffer is of type *floating-point*, you would typically not normalize the distance results, since all this will do is return a value between 0.0 and 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolDraw

Synopsis

Draw specific features of a color-sample, color element, or color matching result.

Syntax

```
void McolDraw(
    AIL_ID GraphContId,         //in
    AIL_ID ContextOrResultId,   //in
    AIL_ID DestImageBufId,      //out
    AIL_INT64 DrawOperation,    //in
    AIL_INT AreaLabel,          //in
    AIL_INT SampleIndexOrLabel, //in
    AIL_INT64 ControlFlag       //in
)
```

Description

This function draws specific features of a selected color-sample (defined in a specified color matching or relative color calibration context), color element (defined in a specified color-sample), or of a color matching result, in the destination image buffer. Drawing operations for color matching results are only available after calling McolMatch().

Color-samples or color elements are drawn using either their triplet values (M_TRIPLET) or their estimated values (for example, the average) taken from a source image (M_IMAGE), depending on how you defined them (McolDefine()).

Parameters

GraphContId

Specifies the identifier of the 2D graphics context to use when drawing. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextOrResultId

Specifies the context or result buffer for which to perform the drawing operation. The context or result buffer must have been previously allocated on the system using McolAlloc() (with M_COLOR_MATCHING or

M_COLOR_CALIBRATION_RELATIVE) or McolAllocResult(), respectively.

DestImageBufId

Specifies the identifier of the destination image buffer in which to draw. The buffer can be any supported 1- or 3-band image buffer. By drawing into its display's overlay buffer, you can also annotate the image non-destructively.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

The required format and size of the destination image depends on the drawing operation. For a color element, you can retrieve the required format and size using McolInquire() (for example, M_SAMPLE_IMAGE_SIZE_X). For a color matching result, you can retrieve this information using McolGetResult() (for example, M_AREA_IMAGE_SIZE_X). When drawing the mosaic of all color elements within a color-sample, you can retrieve the required information using McolInquire() (for example, M_SAMPLE_MOSAIC_SIZE_X).

The bit depth of the image buffer must accommodate all the color-sample or color element data drawn, including the background color, the outlier color, and the outlier label (when applicable).

The numerical value of the color-sample labels, the background pixels, and the outlier pixels and labels should be different so you can easily identify them if they are drawn (M_DRAW...). To modify these values, use McolControl() with M_SAMPLE_LABEL_VALUE, M_BACKGROUND_DRAW_COLOR, M_OUTLIER_DRAW_COLOR and M_OUTLIER_LABEL. Note that a pixel is considered background when it is outside the target areas, while outlier pixels are pixels inside a target area that did not match with any color-sample.

For a color matching result, the background of the destination image is set to the background color (McolControl() with M_BACKGROUND_DRAW_COLOR) that is in effect when you call McolMatch(). If you set a transparent background, the image background remains untouched.

DrawOperation

Specifies the type of drawing operation to perform.

For a selected color-sample, set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For drawing specific features of a color-sample																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_SAMPLE_MOSAIC	Draws the mosaic of all the color elements found in the specified color-sample. If there is only one color element in the sample, M_DRAW_SAMPLE_MOSAIC will essentially perform the same operation as M_DRAW_SAMPLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_SAMPLE_MOSAIC_DONT_CARE	Draws a mosaic mask of all "don't care" pixels in the specified color-sample. All non-zero destination pixels correspond to either a region in-between color data or to a color data mask. All destination pixels set to zero correspond to non-masked color data.	c		e	f	g	a		i		j	k	l	m	n	b

For a selected color element (of a selected color-sample), set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For drawing specific features of a color element																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_SAMPLE	Draws the internal image of the specified color element in the destination image buffer. If you defined a triplet color element, M_DRAW_SAMPLE fills the destination image buffer with the color of the triplet. The triplet values are saturated according to the destination buffer depth. > If you do not specify the index or label of a specific color element, the first color element in the specified color-sample will be drawn.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_SAMPLE_MASK	Draws the mask image of the specified color element in the destination image buffer. You can only perform this operation on M_IMAGE color-samples masked with McolMask(). To inquire if a color-sample has a mask, use McolInquire() with M_SAMPLE_MASKED. > If you do not specify the index or label of a specific color element, the first color element in the specified color-sample will be drawn.	c		e	f	g	a		i		j	k	l	m	n	b

For a color matching result buffer, set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For drawing specific features of a color matching result																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_AREA	Draws the specified area(s) (AreaLabel) of the area identifier image. If you specify a single target area, the background of the destination image (DestImageBufId) is untouched; if you specify M_ALL, the whole area identifier image, including a 0 background, is copied to the destination image. To perform this drawing operation, you must have first enabled McolControl() with M_SAVE_AREA_IMAGE, and called McolMatch().	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_AREA_MATCH_USING_COLOR +	Draws, for each target area, the color of the best-matched color-sample. The background color and outlier color are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_DRAW_COLOR, respectively. To perform this drawing operation, you must have first enabled McolControl() with M_SAVE_AREA_IMAGE and M_GENERATE_SAMPLE_COLOR_LUT, and called McolMatch().	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_AREA_MATCH_USING_LABEL	Draws, for each target area, the label of the best-matched color-sample. The background color and outlier label are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_LABEL, respectively. To perform this drawing operation, you must have first enabled McolControl() with M_SAVE_AREA_IMAGE, and called McolMatch().	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_DISTANCE	Draws the distance between the estimated color (average color) of the target area (for an M_STAT_MIN_DIST operation mode) or the color of the target pixel (for an M_MIN_DIST_VOTE operation mode), and the color of its best-matched color-sample. Operation modes are set with McolSetMethod(). The background color and outlier color are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_DRAW_COLOR, respectively. To perform this drawing operation, you must have first enabled McolControl() with M_GENERATE_DISTANCE_IMAGE, and called McolMatch(). To draw normalized distances, use McolControl() with M_DISTANCE_IMAGE_NORMALIZE before drawing. The outlier color, which is also drawn with M_DRAW_DISTANCE, can be difficult to identify if it is similar to the resulting distance. For example, if the outlier color is 0.0, you wouldn't be able to distinguish it if the resulting distance is also 0.0. In this case, you can use M_DRAW_..._USING_LABEL, which draws the outlier label. M_DRAW_DISTANCE is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_PIXEL_MATCH_USING_COLOR +	Draws, for each pixel in each target area, the color of the color-sample for which each pixel voted. The background color and outlier color are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_DRAW_COLOR, respectively. To perform this drawing operation, you must have first enabled McolControl() with M_GENERATE_PIXEL_MATCH and M_GENERATE_SAMPLE_COLOR_LUT, and called McolMatch(). You must have also used McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b

		c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_PIXEL_MATCH_USING_LABEL	Draws, for each pixel in each target area, the label of the color-sample for which each pixel voted. The background color and outlier label are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_LABEL, respectively. To perform this drawing operation, you must have first enabled McolControl() with M_GENERATE_PIXEL_MATCH, and called McolMatch(). You must also use have used McolSetMethod() with M_MIN_DIST_VOTE.															

For either a color matching context or color matching result, set this parameter to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For drawing specific features of either a color-sample or a color matching result (for color matching)																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_SAMPLE_COLOR_LUT +	Draws the 3-band color-sample label LUT, where the label value of each color-sample is associated with its average color. The average colors are calculated using the color data that you provided when defining the color-sample (McolDefine()). The calculation takes into account all three bands, regardless of the setting of McolControl() with M_BAND_MODE. To perform this operation for color matching results, you must have first enabled McolControl() with M_GENERATE_SAMPLE_COLOR_LUT, and called McolMatch(). M_DRAW_SAMPLE_COLOR_LUT requires that you set the SampleIndexOrLabel parameter to M_ALL or M_DEFAULT. When using M_DRAW_SAMPLE_COLOR_LUT, the destination image buffer (DestImageBufId) must be a 3-band, 8-bit unsigned LUT buffer, with a size of *N* x 1, where *N* is equal to the M_SAMPLE_LUT_SIZE_X inquire type (McolInquire()). You can associate this color-sample label LUT to either a display using MdispLut(), or to an image buffer using MbufControl() with M_ASSOCIATED_LUT.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_DRAW_AREA_MATCH_USING_COLOR; M_DRAW_PIXEL_MATCH_USING_COLOR; M_DRAW_SAMPLE_COLOR_LUT.

You can add the following value to the above-mentioned values to specify whether to invert colors.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to invert colors																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INVERTED_COLORS	Inverts the color of the best-matched color-sample in the specified image.	c		e	f	g	a		i		j	k	l	m	n	b

AreaLabel

Specifies the label of the target area for which to perform the drawing operation, for color matching contexts. For a relative color calibration context, there are no target areas; you must specify M_DEFAULT.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the target area's label or index (for color matching)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default. For a color matching context, the default is the same as M_ALL. For a relative color calibration context, the default indicates that this information is not required.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies all target areas. You should specify M_ALL if you did not use an area identifier image (**M_NULL**) with McolMatch(). This value is only available for color matching contexts.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of the target area, as it appears in the area identifier image. When specifying a specific target area's label, you must use McolControl() with M_SAVE_AREA_IMAGE set to M_ENABLE. This value is only available for color matching contexts.	c		e	f	g	a		i		j	k	l	m	n	b

SampleIndexOrLabel

Specifies the color-sample or color element therein for which to perform the drawing operation. Unless otherwise specified, values apply to both color matching and relative color calibration contexts. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the color-sample or color element																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b

M_REFERENCE_SAMPLE_ITEM(AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to the reference color-sample. This value only applies to relative color calibration contexts (ContextOrResultId).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ElementIndex Specifies the index of a color element (item). The index value must be greater than or equal to 0.															
M_SAMPLE_INDEX(AIL_INT SampleIndex)	Specifies the color-sample's index, in a color context or result.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleIndex Specifies an index value greater than or equal to 0.															
M_SAMPLE_INDEX_ITEM(AIL_INT SampleIndex, AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to a color-sample index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleIndex Specifies the index of the color-sample that holds the color element (item). The index must be greater than or equal to 0.															
	ElementIndex Specifies the index of the color element (item). The index value must be greater than or equal to 0.															
M_SAMPLE_LABEL(AIL_INT SampleLabel)	Specifies the label of a color-sample. To use this value, you must typically enable McolControl() with M_GENERATE_PIXEL_MATCH. However, you need not do this when performing an M_DRAW_AREA_MATCH_USING_LABEL or M_DRAW_AREA_MATCH_USING_COLOR drawing operation.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleLabel Specifies a label value greater than 0.															

M_SAMPLE_LABEL_ITEM(AIL_INT SampleLabel, AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to a color-sample label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleLabel Specifies the label of the color-sample that holds the color element (item). The label value must be greater than 0.															
	ElementIndex Specifies the index of the color element (item). The index value must be greater than or equal to 0.															
M_ALL	Specifies all color-samples. When specifying a color matching context, you can only use M_ALL with M_DRAW_SAMPLE_COLOR_LUT. When specifying a specific color-sample (in either a color matching or relative color calibration context), you can only use M_ALL with M_DRAW_SAMPLE and M_DRAW_SAMPLE_MASK. You can use M_ALL with any operation that a color matching result buffer supports.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFERENCE_SAMPLE	Specifies the reference color-sample. This value only applies to relative color calibration contexts.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolFree

Synopsis

Free a color matching context, a relative color calibration context, or a color matching result buffer.

Syntax

```
void McolFree(
    AIL_ID ContextIdOrResultId //in
)
```

Description

This function deletes the specified context or result buffer, and releases any memory associated with it. If specifying a context, this function deletes all its color-samples. To only delete color-samples in a context, use McolDefine() with M_DELETE.

All contexts and all result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ContextIdOrResultId

Specifies the identifier of the context or result buffer to free. These must have been successfully allocated (with McolAlloc() or McolAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolGetResult

Synopsis

Get the specified type of results from a color matching result buffer.

Syntax

```
void McolGetResult(
    AIL_ID ResultId,                 //in
    AIL_INT AreaLabel,               //in
    AIL_INT ColorSampleIndexOrLabel, //in
    AIL_INT64 ResultType,            //in
    void *ResultArrayPtr             //out
)
```

Description

This function retrieves the results of the specified type from a color matching result buffer. Results are only available after calling McolMatch(), and are organized by target area labels and color-sample indices. You can get results obtained from a specific target area or for all target areas. Similarly, you can get results obtained from a specific color-sample or for all color-samples.

To determine the order of the results, use M_AREA_LABEL_VALUE, and set the AreaLabel parameter to M_ALL and the ColorSampleIndexOrLabel parameter to M_GENERAL.

Parameters

ResultId

Specifies the identifier of the color matching result buffer from which to get results.

AreaLabel

Specifies the label of the target area(s) for which to get results.

Target areas are specified using McolMatch() with the AreaIdentifierImageId parameter. If you set AreaIdentifierImageId to **M_NULL**, you must set the AreaLabel parameter to M_ALL or M_DEFAULT.

Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the target area's label																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to retrieve results for each target area.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to retrieve general results relating to the color matching context. This type of result is not specific to any one target area or color-sample.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0	Specifies a specific target area's label.	c		e	f	g	a		i		j	k	l	m	n	b

ColorSampleIndexOrLabel

Specifies the color-sample(s) for which to get results. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the color-sample																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SAMPLE_INDEX(AIL_INT SampleIndex)	Specifies the index of a color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleIndex Specifies an index value from 0 (inclusive) to the number of defined color-samples minus 1.															
M_SAMPLE_LABEL(AIL_INT SampleLabel)	Specifies the label of a color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleLabel Specifies a label value greater than 0.															
M_ALL	Specifies to retrieve results for each color-sample. Results are ordered according to the color-sample's index value, in increasing order.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to retrieve general results not specific to any one color-sample.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE

- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64

Specifies the address of the first element of the array in which to write the requested information.

This array must be large enough to hold all entries. For example, if you retrieve results for all target areas and color-samples, you must account for the following array size: *NumberOfTargetAreas* x *NumberOfSamples*.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For retrieving general results relating to the color matching context
- For retrieving general target area results
- For retrieving color-sample results

To retrieve general results relating to the color matching context, the ResultType parameter can be set to one of the following values. In this case, you must set the AreaLabel and ColorSampleIndexOrLabel parameters to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving general results relating to the color matching context																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_AREA_IMAGE_SIGN +		Retrieves the data type required for the image buffer in which to draw the target area(s) of the area identifier image (McolDraw() with M_DRAW_AREA). This result is available after you specify an area identifier image with McolMatch(), enable the McolControl() M_SAVE_AREA_IMAGE control type, and find a match.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED	Specifies unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b

M_AREA_IMAGE_SIZE_BAND +	Retrieves the number of surfaces (color bands) required for the image buffer in which to draw the target area(s) of the area identifier image (McolDraw() with M_DRAW_AREA). This result is available after you specify an area identifier image with McolMatch(), enable the McolControl() M_SAVE_AREA_IMAGE control type, and find a match.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
1	Specifies 1 color band.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_IMAGE_SIZE_BIT +	Retrieves the depth per band required for the image buffer in which to draw the target area(s) of the area identifier image (McolDraw() with M_DRAW_AREA). This result is available after you specify an area identifier image with McolMatch(), enable the McolControl() M_SAVE_AREA_IMAGE control type, and find a match.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
1	Specifies 1-bit data depth per band.	c		e	f	g	a		i		j	k	l	m	n	b
8	Specifies 8-bit data depth per band.	c		e	f	g	a		i		j	k	l	m	n	b
16	Specifies 16-bit data depth per band.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_IMAGE_SIZE_X +	Retrieves the width required for the image buffer in which to draw the target area(s) of the area identifier image (McolDraw() with M_DRAW_AREA). This result is available after you specify an area identifier image with McolMatch(), enable the McolControl() M_SAVE_AREA_IMAGE control type, and find a match.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_AREA_IMAGE_SIZE_Y +	Retrieves the height required for the image buffer in which to draw the target area(s) of the area identifier image (McolDraw() with M_DRAW_AREA). This result is available after you specify an area identifier image with McolMatch(), enable the McolControl() M_SAVE_AREA_IMAGE control type, and find a match.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_IMAGE_TYPE +	Retrieves the data type and depth required for the image buffer in which to draw the target area(s) of the area identifier image (McolDraw() with M_DRAW_AREA). This result is available after you specify an area identifier image with McolMatch(), enable the McolControl() M_SAVE_AREA_IMAGE control type, and find a match.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 1	Specifies 1-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies 8-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 16	Specifies 16-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_MATCH_IMAGE_SIZE_X +	Retrieves the width required for the image buffer in which to draw (for each target area) the color or label of the best-matched color-sample (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_COLOR or M_DRAW_AREA_MATCH_USING_LABEL). This result is available after you enable the McolControl() M_SAVE_AREA_IMAGE control type (required for McolDraw()), and find a match.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the width of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_MATCH_IMAGE_SIZE_Y +		Retrieves the height required for the image buffer in which to draw (for each target area) the color or label of the best-matched color-sample (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_COLOR or M_DRAW_AREA_MATCH_USING_LABEL). This result is available after you enable the McolControl() M_SAVE_AREA_IMAGE control type (required for McolDraw()), and find a match.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_IMAGE_SIGN +		Retrieves the data type required for the image buffer in which to draw the distance between the color of the target area (for an M_STAT_MIN_DIST operation mode) or target pixel (for an M_MIN_DIST_VOTE operation mode), and the color of its best-matched color-sample (McolDraw() or McolMatch() with M_DRAW_DISTANCE). Operation modes are set with McolSetMethod(). This result is available after you enable the McolControl() M_GENERATE_DISTANCE_IMAGE control type (required for McolDraw()), and find a match. Note that if you have normalized distances by the maximum distance calculated, using McolControl() with M_DISTANCE_IMAGE_NORMALIZE set to M_MAX_NORMALIZE, you can draw the distance image in an unsigned image buffer. This result is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FLOAT	Specifies floating-point data.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISTANCE_IMAGE_SIZE_BAND +		Retrieves the number of surfaces (color bands) required for the image buffer in which to draw the distance between the color of the target area (for an M_STAT_MIN_DIST operation mode) or target pixel (for an M_MIN_DIST_VOTE operation mode), and the color of its best-matched color-sample (McolDraw() or McolMatch() with M_DRAW_DISTANCE). Operation modes are set with McolSetMethod(). This result is available after you enable the McolControl() M_GENERATE_DISTANCE_IMAGE control type (required for McolDraw()), and find a match. This result is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	1	Specifies 1 color band.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_IMAGE_SIZE_BIT +		Retrieves the depth per band required for the image buffer in which to draw the distance between the color of the target area (for an M_STAT_MIN_DIST operation mode) or target pixel (for an M_MIN_DIST_VOTE operation mode), and the color of its best-matched color-sample (McolDraw() or McolMatch() with M_DRAW_DISTANCE). Operation modes are set with McolSetMethod(). This result is available after you enable the McolControl() M_GENERATE_DISTANCE_IMAGE control type (required for McolDraw()), and find a match. This result is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	32	Specifies 32-bit data depth per band.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISTANCE_IMAGE_SIZE_X +		Retrieves the width required for the image buffer in which to draw the distance between the color of the target area (for an M_STAT_MIN_DIST operation mode) or target pixel (for an M_MIN_DIST_VOTE operation mode), and the color of its best-matched color-sample (McolDraw() or McolMatch() with M_DRAW_DISTANCE). Operation modes are set with McolSetMethod(). This result is available after you enable the McolControl() M_GENERATE_DISTANCE_IMAGE control type (required for McolDraw()), and find a match. This result is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_IMAGE_SIZE_Y +		Retrieves the height required for the image buffer in which to draw the distance between the color of the target area (for an M_STAT_MIN_DIST operation mode) or target pixel (for an M_MIN_DIST_VOTE operation mode), and the color of its best-matched color-sample (McolDraw() or McolMatch() with M_DRAW_DISTANCE). Operation modes are set with McolSetMethod(). This result is available after you enable the McolControl() M_GENERATE_DISTANCE_IMAGE control type (required for McolDraw()), and find a match. This result is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISTANCE_IMAGE_TYPE +	Retrieves the data type and depth required for the image buffer in which to draw the distance between the color of the target area (for an M_STAT_MIN_DIST operation mode) or target pixel (for an M_MIN_DIST_VOTE operation mode), and the color of its best-matched color-sample (McolDraw() or McolMatch() with M_DRAW_DISTANCE). Operation modes are set with McolSetMethod(). This result is available after you enable the McolControl() M_GENERATE_DISTANCE_IMAGE control type (required for McolDraw()), and find a match. Note that if you have normalized distances by the maximum distance calculated, using McolControl() with M_DISTANCE_IMAGE_NORMALIZE set to M_MAX_NORMALIZE, you can draw the distance image in an unsigned image buffer. This result is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOAT + 32	Specifies 32-bit floating-point data.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_DISTANCE +	Retrieves the maximum (largest) color distance among all distances between the target area and all matching color-samples. When the operation mode is M_STAT_MIN_DIST (McolSetMethod()), the largest color distance refers to the maximal distance among all areas that were matched with a color-sample. When the operation mode is M_MIN_DIST_VOTE (McolSetMethod()), the largest color distance refers to the maximal distance among all the pixels that voted for a color-sample. The maximum distance value is used internally when drawing the color distance of the area identifier image (McolDraw() or McolMatch() with M_DRAW_DISTANCE). This result is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_AREAS +	Retrieves the number of target areas in the area identifier image with which you performed the matching operation.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_SAMPLES +	Retrieves the number of color-samples defined in the context.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_MATCH_IMAGE_SIZE_X +	Retrieves the width required for the image buffer in which to draw (for each pixel in each target area) the color or label of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_PIXEL_MATCH_USING_COLOR or M_DRAW_PIXEL_MATCH_USING_LABEL). This result is not available if the operation mode is set to M_HISTOGRAM_MATCHING (McolSetMethod()). This result is available after you enable the McolControl() M_GENERATE_PIXEL_MATCH control type (required for McolDraw()), and find a match. You must also use McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL_MATCH_IMAGE_SIZE_Y +	Retrieves the height required for the image buffer in which to draw (for each pixel in each target area) the color or label of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_PIXEL_MATCH_USING_COLOR or M_DRAW_PIXEL_MATCH_USING_LABEL). This result is not available if the operation mode is set to M_HISTOGRAM_MATCHING (McolSetMethod()). This result is available after you enable the McolControl() M_GENERATE_PIXEL_MATCH control type (required for McolDraw()), and find a match. You must also use McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_COLOR_SIGN +	Retrieves the data type required for the image buffer in which to draw (for each target area or for each pixel in each target area) the color of the best-matched color-sample or the color of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_COLOR or M_DRAW_PIXEL_MATCH_USING_COLOR). This result is available after you enable the McolControl() M_GENERATE_PIXEL_MATCH and M_GENERATE_SAMPLE_COLOR_LUT control types (required for McolDraw()), and find a match. You must also use McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_COLOR_SIZE_BAND +	Retrieves the number of surfaces (color bands) required for the image buffer in which to draw (for each target area or for each pixel in each target area) the color of the best-matched color-sample or the color of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_COLOR or M_DRAW_PIXEL_MATCH_USING_COLOR). This result is available after you enable the McolControl() M_GENERATE_PIXEL_MATCH and M_GENERATE_SAMPLE_COLOR_LUT control types (required for McolDraw()), and find a match. You must also use McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_COLOR_SIZE_BIT +	Retrieves the depth per band, in bits, required for the image buffer in which to draw (for each target area or for each pixel in each target area) the color of the best-matched color-sample or the color of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_COLOR or M_DRAW_PIXEL_MATCH_USING_COLOR). This result is available after you enable the McolControl() M_GENERATE_PIXEL_MATCH and M_GENERATE_SAMPLE_COLOR_LUT control types (required for McolDraw()), and find a match. You must also use McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_COLOR_TYPE +	Retrieves the data type and depth required for the image buffer in which to draw (for each target area or for each pixel in each target area) the color of the best-matched color-sample or the color of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_COLOR or M_DRAW_PIXEL_MATCH_USING_COLOR). This result is available after you enable the McolControl() M_GENERATE_PIXEL_MATCH and M_GENERATE_SAMPLE_COLOR_LUT control types (required for McolDraw()), and find a match. You must also use McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LABEL_SIGN +	Retrieves the data type required for the image buffer in which to draw (for each target area or for each pixel in each target area) the label of the best-matched color-sample or the label of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_LABEL or M_DRAW_PIXEL_MATCH_USING_LABEL).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LABEL_SIZE_BAND +	Retrieves the number of surfaces (color bands) required for the image buffer in which to draw (for each target area or for each pixel in each target area) the label of the best-matched color-sample or the label of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_LABEL or M_DRAW_PIXEL_MATCH_USING_LABEL).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LABEL_SIZE_BIT +	Retrieves the depth per band, in bits, required for the image buffer in which to draw (for each target area or for each pixel in each target area) the label of the best-matched color-sample or the label of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_LABEL or M_DRAW_PIXEL_MATCH_USING_LABEL).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_LABEL_TYPE +	Retrieves the data type and depth required for the image buffer in which to draw (for each target area or for each pixel in each target area) the label of the best-matched color-sample or the label of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_LABEL or M_DRAW_PIXEL_MATCH_USING_LABEL).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve general results relating to a target area, the ResultType parameter can be set to one of the following values. In this case, you must set the AreaLabel parameter to a specific value (or M_ALL), and the ColorSampleIndexOrLabel parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving general target area results																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AREA_LABEL_VALUE +	Retrieves the label of the target areas you used to obtain results. If you do not specify an area identifier image with McolMatch(), the whole target image is labeled as target area 1.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_PIXEL_COUNT +	Retrieves the total number of pixels in the target area.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_BEST_MATCH_INDEX +	Retrieves the index of the target area's best-matched color-sample. If there is none, -1 is returned.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_BEST_MATCH_LABEL +	Retrieves the label of the target area's best-matched color-sample. If no color-samples are matched, the value you have set using McolControl() with M_OUTLIER_LABEL is returned.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIER_COVERAGE +		Retrieves the outlier coverage, which is the percentage of the number of pixels in the target area that did not vote for any color-sample. Since this result is based on pixel votes, you must use McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_RELEVANCE +		Retrieves the relevance score of the target area, as a percentage. This value indicates the significance (relevance) of M_SCORE. In statistics, this is similar to the confidence level. The target area's relevance score is based on the operation mode specified, using McolSetMethod(): • M_STAT_MIN_DIST. *RelevanceScore* = (*BestDistance*)$^{-1}$ / sum (*Distance* $^{-1}$), where *BestDistance* is the distance of the best-matched color-sample, and the sum is over all color-samples that have been matched with the target area. • M_MIN_DIST_VOTE and M_HISTOGRAM_VOTE. *RelevanceScore* = sum(*NumberOfVotes*) / *NumberOfPixelsInTheTargetArea* , where sum(*NumberOfVotes*) refers to the total number of votes accumulated by all color-samples. • M_HISTOGRAM_MATCHING. *RelevanceScore* = (100.0 -*BestScore*)$^{-1}$ / sum ((100.0 -*Score*)$^{-1}$), where *BestScore* refers to the winning sample score, and the sum is over all color-samples.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +		Retrieves the match status of the target area.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FAILURE	Specifies that no color-samples have been matched.	c		e	f	g	a		i		j	k	l	m	n	b

M_SUCCESS	Specifies that at least one color-sample has been matched.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results relating to a color-sample, the ResultType parameter can be set to one of the following values. In this case, you must set the AreaLabel parameter to a specific target area (or M_ALL) and the ColorSampleIndexOrLabel parameter to a specific color-sample (or M_ALL).

To retrieve color-sample results for all color-samples and all areas, set both the AreaLabel and ColorSampleIndexOrLabel parameter to M_ALL, and use M_NUMBER_OF_AREAS and M_NUMBER_OF_SAMPLES to set the size of the result array (*NumberOfTargetAreas* x *NumberOfSamples*). Note that to get a color-sample result, you must first refer to its target area.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving color-sample results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_COLOR_DISTANCE +	Retrieves the color distance (difference) between the target area and the specified color-sample, when using McolSetMethod() with the M_STAT_MIN_DIST operation mode. The significance of the distance value returned depends on the distance type that you have set, using McolSetMethod(). For example, a distance value of 1 when using an M_MAHALANOBIS distance type is not the same as when using an M_MANHATTAN distance type. Note that when matching in the CIELAB source color space, you will get distance results in the CIELAB native range.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_COVERAGE +	Retrieves the proportion of pixels within the target area that voted for the color-sample, as a percentage. Since this result is based on pixel votes, you must use McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LABEL_VALUE +	Retrieves the label of the color-sample corresponding to the specified index.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MATCH_STATUS +		Retrieves whether the specified color-sample fulfills the match conditions, with respect to the distance tolerance and acceptance. To determine if the color-sample is the best-matched color-sample, use M_BEST_MATCH_INDEX or M_BEST_MATCH_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_MATCH	Specifies that the color-sample fulfills the match conditions.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_MATCH	Specifies that the color-sample does not fulfill the match conditions.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PIXEL_COUNT +		Retrieves the number of pixels within the target area that voted for the color-sample, when using McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_SCORE +	Retrieves the match score, as a percentage. This score indicates the similarity between the color of the color-sample and the color of the target area. The color-sample's match score is based on the operation mode specified, using McolSetMethod(): • M_STAT_MIN_DIST. *MatchScore* = [*k* / (*k* + *Distance*)] x [(*MaxDistance* - *Distance*) / *MaxDistance*], where *Distance* refers to the current color-sample distance and *MaxDistance* refers to the maximum distance possible (in the color space). • M_MIN_DIST_VOTE and M_HISTOGRAM_VOTE. *MatchScore* = *NumberOfVotes* / sum (*NumberOfVotes*), where *NumberOfVotes* refers to the current color-sample's number of votes, and the sum is over the number of votes for all color-samples. • M_HISTOGRAM_MATCHING. *MatchScore* refers to a measure of similarity between sample and target histograms.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For retrieving general results relating to the color matching context
- For retrieving general target area results
- For retrieving color-sample results

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.															

	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For retrieving general target area results, For retrieving color-sample results.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
ResultType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS		Retrieves the required array size (number of elements) to store the returned values.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For retrieving general results relating to the color matching context
- For retrieving general target area results
- For retrieving color-sample results

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

Some results might be altered if they do not fit numerically in the data type you specify. For example, if you cast a result value of 300 to a *char*, information will be lost. The default data type, *double*, will never result in lost information.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]* **|** *address of an AIL_INT16*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolInquire

Synopsis

Inquire information about a color matching or relative color calibration context, color-sample, color element, or a color matching result.

Syntax

```
AIL_INT McolInquire(
    AIL_ID ContextOrResultId, //in
    AIL_INT IndexOrLabel,     //in
    AIL_INT64 InquireType,    //in
    void *UserVarPtr          //out
)
```

Description

This function inquires information about a specified color matching or relative color calibration context, a color-sample contained within the context, a color element contained within a specified color-sample, or a color matching result buffer.

If the inquired setting is set to M_DEFAULT (for example, in McolControl()), **McolInquire**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

Parameters

ContextOrResultId

Specifies the identifier of the context or result buffer about which to inquire information. Both the context and the result buffer must have been previously allocated on the required system using McolAlloc() (with M_COLOR_MATCHING or M_COLOR_CALIBRATION_RELATIVE) or McolAllocResult().

IndexOrLabel

Specifies what to inquire.

For inquiring about a context or result, set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a context or result																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default. If ContextOrResultId is set to a context, the default is the same as M_CONTEXT. If ContextOrResultId is set to a color matching result, the default is the same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Inquires information about a global setting of either a color matching or relative color calibration context, depending on the ContextOrResultId parameter setting.	c		e	f	g	a		i		j	k	l	m	n	b

M_GENERAL	Inquires information about a setting of a color matching result buffer. In this case, ContextOrResultId must specify a color matching result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

For inquiring about a color-sample or color element therein, set this parameter to one of the following values. Unless otherwise specified, the ContextOrResultId parameter can specify a color matching or relative color calibration context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a color-sample or color element																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_REFERENCE_SAMPLE_ITEM(AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to the reference color-sample. This value only applies to relative color calibration contexts (ContextOrResultId).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ElementIndex Specifies the index of a color element (item). The index value must be greater than or equal to 0.															
M_SAMPLE_INDEX(AIL_INT SampleIndex)	Specifies the index of a color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleIndex Specifies an index value greater than or equal to 0.															
M_SAMPLE_INDEX_ITEM(AIL_INT SampleIndex, AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to a color-sample index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleIndex Specifies the index of the color-sample that holds the color element (item). The index must be greater than or equal to 0.															
	ElementIndex Specifies the index of the color element (item). The index value must be greater than or equal to 0.															

M_SAMPLE_LABEL(AIL_INT SampleLabel)	Specifies the label of a color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleLabel Specifies a label value greater than 0.															
M_SAMPLE_LABEL_ITEM(AIL_INT SampleLabel, AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to a color-sample label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleLabel Specifies the label of the color-sample that holds the color element (item). The label value must be greater than 0.															
	ElementIndex Specifies the index of the color element (item). The index value must be greater than or equal to 0.															
M_REFERENCE_SAMPLE	Specifies the reference color-sample. This value only applies to relative color calibration contexts (ContextOrResultId).	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **McolInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the

UserVarPtr parameter.

- For inquiring general settings about a context
- For inquiring color matching context settings that are used for color space encoding
- For inquiring about a general setting of a color-sample defined in a context
- For inquiring about a general setting of a color element (item) defined in a context
- For inquiring information about the PCA of a color-sample in a color matching context
- For inquiring context or result buffer settings

To inquire about general settings of a context, set the InquireType parameter to one of the following values. In this case, you must set the IndexOrLabel parameter to M_CONTEXT and the ContextOrResultId parameter to a color matching or relative color calibration context, as required by the setting. Unless otherwise specified, the following settings only apply to color matching contexts.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring general settings about a context																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ACCEPTANCE +	Inquires the acceptance level for the color-sample's score. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCEPTANCE_RELEVANCE +	Inquires the acceptance level for the target area's relevance score. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies an acceptable relevance score, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_BACKGROUND_DRAW_COLOR +	Inquires the background pixel color, when applicable. This value applies to McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_COLOR, M_DRAW_PIXEL_MATCH_USING_COLOR, or M_DRAW_DISTANCE. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_TRANSPARENT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRANSPARENT	Specifies not to change the background. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Byte-encoded RGB value	Specifies an encoded RGB value. To verify if the value is a byte-encoded RGB value, use the **M_IS_RGB888** macro. To retrieve the R, G, and B bands, use the **M_RGB888_R**, **M_RGB888_G**, and **M_RGB888_B** macros.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies a grayscale value for the background pixels, as a *double* or *integer*. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BAND_MODE +		Inquires the color bands to use when performing McolMatch(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ALL_BANDS.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL_BANDS	Specifies to use all bands.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BAND_0	Specifies to use band 0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BAND_0 + M_COLOR_BAND_1	Specifies to use bands 0 and 1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BAND_0 + M_COLOR_BAND_2	Specifies to use bands 0 and 2.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BAND_1	Specifies to use band 1.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BAND_1 + M_COLOR_BAND_2	Specifies to use bands 1 and 2.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_BAND_2	Specifies to use band 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_COMPUTE_OPTION +		Inquires whether the relative color calibration was performed using pixel values or color statistics. This inquire type is only available for relative color calibration contexts. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_COMPUTE_ITEM_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPUTE_ITEM_PIXELS	Specifies that Aurora Imaging Library uses all pixel values (in the reference color-sample and the color-samples) to generate the color information that the operation requires.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPUTE_ITEM_STAT	Specifies that Aurora Imaging Library uses color statistics (in the reference color-sample and the color-samples) to generate the color information that the operation requires. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_INTENT +		Inquires the transformation precision with which to perform relative color calibration. This inquire type is only available for relative color calibration contexts. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default color calibration intent. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BALANCE	Specifies that the operation considers a moderate amount of color information to establish the mapping. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GENERALIZATION	Specifies that the operation considers a minimal amount of color information to establish the mapping. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PRECISION	Specifies that the operation considers a high amount of color information to establish the mapping. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_MODE +		Inquires the transformation operation with which to perform relative color calibration. This inquire type is only available for relative color calibration contexts. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_HISTOGRAM_BASED.	c		e	f	g	a		i		j	k	l	m	n	b

M_COLOR_TO_COLOR	Specifies that a color-sample's mapping is based on transforming its color to the color of the reference color-sample on a point-to-point basis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_MEAN_VARIANCE	Specifies that a color-sample's mapping is based on transforming its mean and variance (standard deviation) to resemble the mean and variance of the reference color-sample. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HISTOGRAM_BASED	Specifies that a color-sample's mapping is based on using enhanced color distribution information to transform its color data to better resemble the color data of the reference color-sample. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_SPACE +	Inquires the context's color space. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_RGB.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIELAB	Specifies a CIELAB (or LAB) color space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HSL	Specifies an HSL color space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB	Specifies an RGB color space. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT_TYPE +	Inquires the type of context used.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CALIBRATION_RELATIVE	Allocates a relative color calibration context for use with McolTransform().	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_MATCHING	Allocates a color matching context for use with McolMatch().	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVERSION_GAMMA +	Inquires whether gamma correction is performed when converting colors before the match operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to remove gamma correction. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to remove gamma correction. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CONVERSION_MODE +		Inquires the mode used to internally convert colors before the matching operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_HSL	Specifies to internally convert RGB colors to HSL before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies no internal conversion.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_IMAGE_NORMALIZE +		Inquires the normalization factor when drawing or generating M_DRAW_DISTANCE, using McolDraw() or McolMatch(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NO_NORMALIZE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX_NORMALIZE	Specifies to normalize distance values according to the greatest calculated distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_NORMALIZE	Specifies no normalization. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies to normalize distance values according to the specified normalization value. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_DISTANCE_TOLERANCE_MODE +	Inquires the strategy used to calculate the tolerance for an acceptable color distance between the color-sample and a target area. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ABSOLUTE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ABSOLUTE	Specifies that the M_DISTANCE_TOLERANCE value will be untouched and used as the acceptable distance tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
M_RELATIVE	Specifies a tolerance strategy based on the relative distances between pairs of color-samples. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_STDDEV	Specifies a tolerance strategy based on the color-sample's standard deviation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISTANCE_TYPE +	Inquires the type of distance calculated when matching colors. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default distance type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CIE94_GRAPHIC_ARTS	Specifies the CIE94 color distance with weighting factors pertaining to the graphic arts industry, as established by the CIE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CIE94_TEXTILE	Specifies the CIE94 color distance with weighting factors pertaining to the textile industry, as established by the CIE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CIEDE2000	Specifies the CIEDE2000 color distance, as established by the CIE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CMC_ACCEPTABILITY	Specifies the CMC(*l*: *c*) color distance that has been adjusted for acceptability evaluation, as established by the CIE. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_CMC_PERCEPTIBILITY	Specifies the CMC(*l*: *c*) color distance that has been adjusted for perceptibility evaluation, as established by the CIE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DELTA_E	Specifies a Delta-E color distance, as defined by the International Commission on Illumination (CIE). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_EUCLIDEAN	Specifies a Euclidean color distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAHALANOBIS	Specifies a Mahalanobis color distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MANHATTAN	Specifies a Manhattan color distance. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies that no distance calculations are used. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERATE_DISTANCE_IMAGE +		Inquires whether the distance image was generated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to generate the image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to generate the image.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERATE_PIXEL_MATCH +		Inquires whether the image containing the color-sample label for which each pixel voted was generated. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to generate the image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to generate the image.	c		e	f	g	a		i		j	k	l	m	n	b

M_GENERATE_SAMPLE_COLOR_LUT +	Inquires whether the color-sample label LUT was generated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to generate the LUT.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to generate the LUT.	c		e	f	g	a		i		j	k	l	m	n	b
M_MATCH_MODE +	Inquires the operation mode used when matching colors. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_STAT_MIN_DIST.	c		e	f	g	a		i		j	k	l	m	n	b
M_HISTOGRAM_MATCHING	Specifies a minimum distance operation, based on color histograms. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HISTOGRAM_VOTE	Specifies a match operation based on histogram voting. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_DIST_VOTE	Specifies a minimum distance operation, based on pixel voting. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_DIST	Specifies a minimum distance operation, based on pixel statistics. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFICATION_COUNT +	Inquires the current value of the modification counter. The modification counter is increased by one each time settings for the context are modified. Although you cannot identify the modification counter's contents, you can compare them throughout your application to inquire if the context has been altered. If the modification counter has changed you can, for example, prompt the user to save before closing the application. This inquire type is available for color matching and relative color calibration contexts.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the current value of the modification counter.	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_BINS_BAND_0 +	Inquires the number of histogram bins for color band 0. Relevant only when matching colors using McolSetMethod() with M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of histogram bins for color band 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_BINS_BAND_1 +	Inquires the number of histogram bins for color band 1. Relevant only when matching colors using McolSetMethod() with M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Number of histogram bins for color band 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_BINS_BAND_2 +	Inquires the number of histogram bins for color band 2. Relevant only when matching colors using McolSetMethod() with M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Number of histogram bins for color band 2.	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_SAMPLES +	Inquires the number of color-samples defined in the context (McolDefine()). This inquire type is available for color matching and relative color calibration contexts. For relative color calibration contexts, the number of color-samples does not include the reference color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of color-samples.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUTLIER_DRAW_COLOR +	Inquires the color used for outlier pixels when drawing the color of the best-matched color-sample, the color of the color-sample for which each pixel voted, or the distance between the color of the target area/target pixel and the color of its best-matched color-sample (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_COLOR, M_DRAW_PIXEL_MATCH_USING_COLOR or M_DRAW_DISTANCE). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSPARENT	Specifies not to change the value of the outlier pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Byte-encoded RGB value	Specifies an encoded RGB value. To verify if the value is a byte-encoded RGB value, use the **M_IS_RGB888** macro. To retrieve the R, G, and B bands, use the **M_RGB888_R**, **M_RGB888_G**, and **M_RGB888_B** macros.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies a grayscale value for the outlier pixels, as a *double* or *integer*. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_OUTLIER_LABEL +	Inquires the label used for outlier pixels when drawing the label of the best-matched color-sample or the label of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_LABEL or M_DRAW_PIXEL_MATCH_USING_LABEL). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of the outlier pixels, as an *integer*. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +	Inquires whether the context is preprocessed. The context must be preprocessed (using McolPreprocess()) before calling McolMatch() or McolTransform(). After certain settings of the context are changed (for example, with McolControl() or McolDefine()), this inquire type will indicate that the context is no longer in its preprocessed state. This inquire type is available for color matching and relative color calibration contexts.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the context is not preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the context is preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LABEL_SIZE_BIT +	Inquires the depth per band required for the image buffer in which to draw the label of the best-matched color-sample or the label of the color-sample for which each pixel voted (McolDraw() or McolMatch() with M_DRAW_AREA_MATCH_USING_LABEL or M_DRAW_PIXEL_MATCH_USING_LABEL).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

1	Specifies that the data depth is 1 bit per band.	c		e	f	g	a		i		j	k	l	m	n	b
8	Specifies that the data depth is 8 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b
16	Specifies that the data depth is 16 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b
32	Specifies that the data depth is 32 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LUT_SIGN +	Inquires the data type required for the image buffer in which to draw the 3-band color-sample label LUT (McolDraw() with M_DRAW_SAMPLE_COLOR_LUT). This LUT associates the label value of each color-sample with its average color.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the data type is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LUT_SIZE_BAND +	Inquires the number of color bands that are required for the image buffer in which to draw the 3-band color-sample label LUT (McolDraw() with M_DRAW_SAMPLE_COLOR_LUT). This LUT associates the label value of each color-sample with its average color.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
3	Specifies that 3 color bands are required.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LUT_SIZE_BIT +	Inquires the depth per band required for the image buffer in which to draw the 3-band color-sample label LUT (McolDraw() with M_DRAW_SAMPLE_COLOR_LUT). This LUT associates the label value of each color-sample with its average color.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
8	Specifies that the data depth is 8 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_LUT_SIZE_X +	Inquires the width required for the image buffer in which to draw the 3-band color-sample label LUT (McolDraw() with M_DRAW_SAMPLE_COLOR_LUT). This LUT associates the label value of each color-sample with its average color.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LUT_SIZE_Y +	Inquires the height required for the image buffer in which to draw the 3-band color-sample label LUT (McolDraw() with M_DRAW_SAMPLE_COLOR_LUT). This LUT associates the label value of each color-sample with its average color.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the image buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_LUT_TYPE +	Inquires the data type and depth required for the image buffer in which to draw the 3-band color-sample label LUT (McolDraw() with M_DRAW_SAMPLE_COLOR_LUT). This LUT associates the label value of each color-sample with its average color.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies that the data is 8-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_AREA_IMAGE +	Inquires whether the area identifier image was saved. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to save the image.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to save the image.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about color matching context settings that are used for color space encoding, set the InquireType parameter to one of the following values. In this case, you must set the IndexOrLabel parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring color matching context settings that are used for color space encoding																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_ENCODING +	Inquires the color space encoding. SET																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_8BIT.	c		e	f	g	a		i		j	k	l	m	n	b
M_8BIT	Specifies an internally defined color space encoding for 8-bit images.	c		e	f	g	a		i		j	k	l	m	n	b
M_10BIT	Specifies an internally defined color space encoding for 10-bit images.	c		e	f	g	a		i		j	k	l	m	n	b
M_12BIT	Specifies an internally defined color space encoding for 12-bit images.	c		e	f	g	a		i		j	k	l	m	n	b
M_14BIT	Specifies an internally defined color space encoding for 14-bit images.	c		e	f	g	a		i		j	k	l	m	n	b
M_16BIT	Specifies an internally defined color space encoding for 16-bit images.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies that the color space encoding is defined using M_OFFSET_BAND_n and M_SCALE_BAND_n.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_BAND_0 +	Inquires the offset for the color space encoding, for band 0. This inquire is only applicable if you use McolControl() with M_ENCODING set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the offset, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_BAND_1 +	Inquires the offset for the color space encoding, for band 1. This inquire is only applicable if you use McolControl() with M_ENCODING set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the offset, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_BAND_2 +	Inquires the offset for the color space encoding, for band 2. This inquire is only applicable if you use McolControl() with M_ENCODING set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the offset, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCALE_BAND_0 +	Inquires the scale for the color space encoding, for band 0. This inquire is only applicable if you use McolControl() with M_ENCODING set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the scale, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCALE_BAND_1 +	Inquires the scale for the color space encoding, for band 1. This inquire is only applicable if you use McolControl() with M_ENCODING set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the scale, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCALE_BAND_2 +	Inquires the scale for the color space encoding, for band 2. This inquire is only applicable if you use McolControl() with M_ENCODING set to M_USER_DEFINED. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the scale, as a *double*.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a general setting of a color-sample defined in a context, set the InquireType parameter to one of the following values. In this case, you must set the IndexOrLabel parameter to the index or label of a color-sample (not an element of a color-sample). Unless otherwise specified, the following settings apply to color matching and relative color calibration contexts.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a general setting of a color-sample defined in a context																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_DISTANCE_TOLERANCE +		Inquires the acceptable tolerance for the color distance between the color-sample and a target area. This inquire type is only available for color matching contexts. SET														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies the tolerance automatically. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies an infinite tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the tolerance value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HISTOGRAM_FREQUENCY_THRESHOLD +		Inquires the minimal frequency, in pixels, for a histogram bin to count in the matching process. This inquire type is only available for color matching contexts. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1 pixel.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the minimal frequency, as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b

M_MATCH_SAMPLE_COLOR_BAND_0 +		Inquires the band 0 value of the color-sample that will be used with McolMatch(). If you specified a color conversion mode using McolSetMethod(), M_MATCH_SAMPLE_COLOR_BAND_0 returns the converted value. To return the unconverted value, use M_SAMPLE_AVERAGE_COLOR_BAND_0. For example, if you are using RGB images in an RBG color matching context, and you specify an HSL conversion mode (McolSetMethod()), M_MATCH_SAMPLE_COLOR_BAND_0 returns the average value for band 0 (hue) of the HSL color (as used by the match operation), while M_SAMPLE_AVERAGE_COLOR_BAND_0 returns the value for band 0 (red) of the RGB color (as defined in McolDefine()). If you have not performed a color conversion, M_MATCH_SAMPLE_COLOR_BAND_0 returns the same value as M_SAMPLE_AVERAGE_COLOR_BAND_0. This inquire type is only available for color matching contexts. This inquire type is only available if you have selected the band (McolControl() with M_BAND_MODE) and have preprocessed the context (McolPreprocess()). This inquire type is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the band 0 value of the color-sample.	c		e	f	g	a		i		j	k	l	m	n	b

M_MATCH_SAMPLE_COLOR_BAND_1 +	Inquires the band 1 value of the color-sample that will be used with McolMatch(). For more information, see M_MATCH_SAMPLE_COLOR_BAND_0. This inquire type is only available for color matching contexts. This inquire type is only available if you have selected the band (McolControl() with M_BAND_MODE) and have preprocessed the context (McolPreprocess()). This inquire type is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the band 1 value of the color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
M_MATCH_SAMPLE_COLOR_BAND_2 +	Inquires the band 2 value of the color-sample that will be used with McolMatch(). For more information, see M_MATCH_SAMPLE_COLOR_BAND_0. This inquire type is only available for color matching contexts. This inquire type is only available if you have selected the band (McolControl() with M_BAND_MODE) and have preprocessed the context (McolPreprocess()). This inquire type is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the band 2 value of the color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_COLOR_ITEMS +	Inquires the number of color elements items in the specified color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the number of color elements in the color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_8BIT_AVERAGE_COLOR_BAND_0 +		Inquires the average color of the color-sample for band 0 (as an 8-bit value). This inquire type is useful for displaying the sample's color. This value is not used in the operation (for example, McolMatch(). This inquire type is only available for color matching contexts. This inquire type is only available if you have preprocessed the context (McolPreprocess()).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the average color of the color-sample for band 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_8BIT_AVERAGE_COLOR_BAND_1 +		Inquires the average color of the color-sample for band 1 (as an 8-bit value). This inquire type is useful for displaying the sample's color. This value is not used in the operation (for example, McolMatch(). This inquire type is only available for color matching contexts. This inquire type is only available if you have preprocessed the context (McolPreprocess()).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the average color of the color-sample for band 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_8BIT_AVERAGE_COLOR_BAND_2 +		Inquires the average color of the color-sample for band 2 (as an 8-bit value). This inquire type is useful for displaying the sample's color. This value is not used in the operation (for example, McolMatch(). This inquire type is only available for color matching contexts. This inquire type is only available if you have preprocessed the context (McolPreprocess()).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the average color of the color-sample for band 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_ABSOLUTE_TOLERANCE +		Inquires the maximum distance a color-sample can have for it to be matched. To retrieve this information, you must first preprocess the context, using McolPreprocess(). This inquire type is only available for color matching contexts.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies that the maximum distance is infinite. This value is returned when only one color-sample has been defined.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the maximum distance.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_AVERAGE_COLOR_BAND_0 +	Inquires the band 0 value of the color-sample, as it was defined using McolDefine(). For color-samples defined from an image (M_IMAGE), the band 0 value returned refers to the estimated version of the color (for example, the average). If you specified a color conversion mode before the match operation using McolSetMethod(), M_SAMPLE_AVERAGE_COLOR_BAND_0 returns the unconverted value. To return the converted value, use M_MATCH_SAMPLE_COLOR_BAND_0. For example, if you are using RGB images in an RBG color matching context, and you specify an HSL conversion mode (McolSetMethod()), M_SAMPLE_AVERAGE_COLOR_BAND_0 returns the value for band 0 (red) of the RGB color (as defined in McolDefine()), while M_MATCH_SAMPLE_COLOR_BAND_0 returns the average value for band 0 (hue) of the HSL color (as used by the match operation). If you have not performed a color conversion, M_SAMPLE_AVERAGE_COLOR_BAND_0 returns the same value as M_MATCH_SAMPLE_COLOR_BAND_0. This inquire type is only available for color matching contexts. This inquire type is only available if you have selected the band (McolControl() with M_BAND_MODE) and have preprocessed the context (McolPreprocess()).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the band 0 value of the color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_AVERAGE_COLOR_BAND_1 +	Inquires the band 1 value of the color-sample, as it was defined using McolDefine(). For more information, see M_SAMPLE_AVERAGE_COLOR_BAND_0. This inquire type is only available for color matching contexts. This inquire type is only available if you have selected the band (McolControl() with M_BAND_MODE) and have preprocessed the context (McolPreprocess()).	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c	e	f	g	a	i	j	k	l	m	n	b
	Value	Specifies the band 1 value of the color-sample.	c	e	f	g	a	i	j	k	l	m	n	b
M_SAMPLE_AVERAGE_COLOR_BAND_2 +		Inquires the band 2 value of the color-sample, as it was defined using McolDefine(). For more information, see M_SAMPLE_AVERAGE_COLOR_BAND_0. This inquire type is only available for color matching contexts. This inquire type is only available if you have selected the band (McolControl() with M_BAND_MODE) and have preprocessed the context (McolPreprocess()).	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c	e	f	g	a	i	j	k	l	m	n	b
	Value	Specifies the band 2 value of the color-sample.	c	e	f	g	a	i	j	k	l	m	n	b
M_SAMPLE_LABEL_VALUE +		Inquires the current label of the color-sample. SET SET	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c	e	f	g	a	i	j	k	l	m	n	b
	Value	Specifies the current label of the color-sample.	c	e	f	g	a	i	j	k	l	m	n	b
M_SAMPLE_MOSAIC_DONT_CARE_SIGN +		Inquires the data type of the color-sample's mask mosaic buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c	e	f	g	a	i	j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c	e	f	g	a	i	j	k	l	m	n	b
	M_UNSIGNED	Specifies that the color-sample's mask mosaic buffer is of data type unsigned.	c	e	f	g	a	i	j	k	l	m	n	b
M_SAMPLE_MOSAIC_DONT_CARE_SIZE_BAND +		Inquires the number of bands of the color-sample's mask mosaic buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c	e	f	g	a	i	j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	1	Specifies that the color-sample's mask mosaic buffer has 1 band.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MOSAIC_DONT_CARE_SIZE_BIT +		Inquires the depth per band of the color-sample's mask mosaic buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	1	Specifies that the data depth is 1 bit per band.	c		e	f	g	a		i		j	k	l	m	n	b
	8	Specifies that the data depth is 8 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MOSAIC_DONT_CARE_SIZE_X +		Inquires the X-size (width) of the color-sample's mask mosaic buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-size (width) of the color-sample's mask mosaic buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MOSAIC_DONT_CARE_SIZE_Y +		Inquires the Y-size (height) of the color-sample's mask mosaic buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-size (height) of the color-sample's mask mosaic buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MOSAIC_DONT_CARE_TYPE +		Inquires the data type and depth of the color-sample's mask buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 1	Specifies that the data is 1-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 8	Specifies that the data is 8-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MOSAIC_SIGN +		Inquires the data type of the color-sample's mosaic buffer. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED	Specifies that the color-sample's mosaic buffer is of data type unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MOSAIC_SIZE_BAND +		Inquires the number of bands of the color-sample's mosaic buffer. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	3	Specifies that the color-sample's mosaic buffer has 3 bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MOSAIC_SIZE_BIT +		Inquires the depth per band of the color-sample's mosaic buffer. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	8	Specifies that the data depth is 8 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b
	16	Specifies that the data depth is 16 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_MOSAIC_SIZE_X +	Inquires the X-size (width) of the color-sample's mosaic buffer. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-size (width) of the color-sample's mosaic buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MOSAIC_SIZE_Y +	Inquires the Y-size (height) of the color-sample's mosaic buffer. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-size (height) of the color-sample's mosaic buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MOSAIC_TYPE +	Inquires the type of the color-sample's mosaic buffer. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies that the data is 8-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 16	Specifies that the data is 16-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_TYPE +	Inquires the type of the color-sample. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE	Specifies that the color-sample was added from an image.	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRIPLET	Specifies that the color-sample was added from three explicit color band values (RGB, HSL, or CIELAB).	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a general setting of a color element defined in a color-sample, set the InquireType parameter to one of the following values. In this case, you must set the IndexOrLabel parameter to the index or label of a color-sample element.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a general setting of a color element (item) defined in a context																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_SAMPLE_COLOR_BAND_0 +	Inquires the color value for band 0 of a triplet color element. This inquire type is only available for color elements that you define using McolDefine() with M_TRIPLET. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the color value for band 0 of a triplet color element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_COLOR_BAND_1 +	Inquires the color value for band 1 of a triplet color element. This inquire type is only available for color elements that you define using McolDefine() with M_TRIPLET. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the color value for band 1 of a triplet color element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_COLOR_BAND_2 +	Inquires the color value for band 2 of a triplet color element. This inquire type is only available for color elements that you define using McolDefine() with M_TRIPLET. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the color value for band 2 of a triplet color element.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_IMAGE_SIGN +		Inquires the data type of the image from which the color element was defined. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED	Specifies that the data type is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_IMAGE_SIZE_BAND +		Inquires the number of color bands of the image buffer from which the color element was defined. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	3	Specifies that the image buffer has 3 color bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_IMAGE_SIZE_BIT +		Inquires the depth per band of the image buffer from which the color element was defined. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	8	Specifies that the data depth is 8 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b
	16	Specifies that the data depth is 16 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_IMAGE_SIZE_X +		Inquires the width of the image from which the color element was defined. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width of the image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_IMAGE_SIZE_Y +	Inquires the height of the image from which the color element was defined. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE. This inquire type is available for color matching and relative color calibration contexts.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_IMAGE_TYPE +	Inquires the data type and depth of the image from which the color element was defined. This inquire type is only available for color elements that you define using McolDefine() with M_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies that the data is 8-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 16	Specifies that the data is 16-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MASK_SIGN +	Inquires the data type of the color element's mask buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the data type is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MASK_SIZE_BAND +	Inquires the number of color bands of the color element's mask buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
1	Specifies that there is 1 band in the color element's mask buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_MASK_SIZE_BIT +		Inquires the depth per band of the color element's mask buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	1	Specifies that the data depth is 1 bit per band.	c		e	f	g	a		i		j	k	l	m	n	b
	8	Specifies that the data depth is 8 bits per band.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MASK_SIZE_X +		Inquires the width of the color element's mask buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width of the color element's mask buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MASK_SIZE_Y +		Inquires the height of the color element's mask buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height of the color element's mask buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_MASK_TYPE +		Inquires the data type and depth of the color element's mask buffer. This inquire type is only available if you have defined a mask for the color-sample, using McolMask().	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 1	Specifies that the data is 1-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 8	Specifies that the data is 8-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_MASKED +	Inquires whether a mask for the color element was defined.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that a mask was not defined.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that a mask was defined.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire information about the principal component analysis (PCA) of a color-sample in a color matching context, set the InquireType parameter to one of the following values. In this case, you must set the IndexOrLabel parameter to M_SAMPLE_INDEX() or M_SAMPLE_LABEL(); you must have also preprocessed the context, using McolPreprocess().

A PCA is a mathematical analysis which, when applied to a color image's data, results in vectors pointing towards the direction of maximum color variance. Each vector is considered a principal component, and there are as many principal components as there are color bands. Since color-samples are based on three bands, the PCA results in three vectors about which to inquire. These PCA values are not applied in the actual operation (for example, McolMatch() and should only be inquired by advanced users who might, for example, want to perform some statistical analysis. Typically, color operations such as McolMatch() and McolProject() use what is referred to as the first principal component, which is the strongest principal component computed from a PCA. If no principal component is the strongest, one is arbitrarily considered the first principal component.

Aurora Imaging Library calculates the PCA values according to the context's source color space; these values are not available when using M_HSL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring information about the PCA of a color-sample in a color matching context																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_SAMPLE_PCA_VAR1 +	Inquires the variance in the direction of the color-sample's first principal component vector.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the variance.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VAR2 +	Inquires the variance in the direction of the color-sample's second principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the variance.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAMPLE_PCA_VAR3 +	Inquires the variance in the direction of the color-sample's third principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the variance.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VECTOR1_1 +	Inquires the first element of the color-sample's first principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vector element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VECTOR1_2 +	Inquires the second element of the color-sample's first principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vector element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VECTOR1_3 +	Inquires the third element of the color-sample's first principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vector element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VECTOR2_1 +	Inquires the first element of the color-sample's second principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vector element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VECTOR2_2 +	Inquires the second element of the color-sample's second principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vector element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VECTOR2_3 +	Inquires the third element of the color-sample's second principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vector element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VECTOR3_1 +	Inquires the first element of the color-sample's third principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vector element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VECTOR3_2 +	Inquires the second element of the color-sample's third principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vector element.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAMPLE_PCA_VECTOR3_3 +	Inquires the third element of the color-sample's third principal component vector.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vector element.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about context or result buffer settings, the InquireType parameter can be set to one of the following values. When inquiring about a context, set the IndexOrLabel parameter to M_CONTEXT. When inquiring about a result buffer, set the IndexOrLabel parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring context or result buffer settings																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_OWNER_SYSTEM +	Inquires the identifier of the system on which either the context or result buffer has been allocated. SET SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b

	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For inquiring general settings about a context
- For inquiring color matching context settings that are used for color space encoding
- For inquiring about a general setting of a color-sample defined in a context
- For inquiring information about the PCA of a color-sample in a color matching context
- For inquiring context or result buffer settings

the values listed in all the parameter association tables except **For inquiring about a general setting of a color element (item) defined in a context**.

You can add the following value to the above-mentioned values to determine the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value of an inquire type																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DEFAULT		Inquires the default value of the specified inquire type.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For inquiring general settings about a context
- For inquiring color matching context settings that are used for color space encoding
- For inquiring about a general setting of a color-sample defined in a context
- For inquiring information about the PCA of a color-sample in a color matching context
- For inquiring context or result buffer settings

the values listed in all the parameter association tables except **For inquiring about a general setting of a color element (item) defined in a context**.

You can add the following value to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring whether an inquire type is supported																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																

M_SUPPORTED		Inquires whether the specified inquire type is supported.														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For inquiring general settings about a context
- For inquiring color matching context settings that are used for color space encoding
- For inquiring about a general setting of a color-sample defined in a context
- For inquiring about a general setting of a color element (item) defined in a context
- For inquiring information about the PCA of a color-sample in a color matching context
- For inquiring context or result buffer settings

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Some inquired information might be altered if it does not fit numerically in the data type that you specify. For example, if you cast a value of 300 to a *char*, information will be lost. The default data type, *double*, will never result in lost information.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE		Casts the requested information to an *AIL_DOUBLE*.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT		Casts the requested information to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID		Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT		Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT16*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolMask

Synopsis

Mask regions of a color element in a specified color-sample.

Syntax

```
void McolMask(
    AIL_ID ContextId,                //out
    AIL_INT ColorSampleIndexOrLabel, //in
    AIL_ID MaskBufferId,             //in
    AIL_INT64 MaskType,              //in
    AIL_INT64 ControlFlag            //in
)
```

Description

This function allows you to apply masks to specific color elements of the specified color-sample in the color matching or relative color calibration context. To apply a mask to a color-sample with only one element, you must have added the color-sample to the context using McolDefine() with M_IMAGE. To apply a mask to a specific color element of the specified color-sample, you must have added the color element to a color-sample using McolDefine() with M_ADD_COLOR_TO_SAMPLE.

With each mask, you can set pixels in the specified color element to a "don't care" state. Aurora Imaging Library does not consider "don't care" pixels in color-samples when you call McolPreprocess(), McolMatch(), or McolTransform().

Parameters

ContextId

Specifies the context in which the color-sample containing the color element to be masked is located. You must first allocate the color matching or relative color calibration context on the system using McolAlloc() with M_COLOR_MATCHING or M_COLOR_CALIBRATION_RELATIVE.

ColorSampleIndexOrLabel

Specifies the color element for which to create a mask. Unless otherwise specified, values apply to color matching and relative color calibration contexts. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the color element																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_REFERENCE_SAMPLE_ITEM(AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to the reference color-sample. This value only applies to relative color calibration contexts (ContextId).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

	ElementIndex Specifies the index of a color element (item). The index must be greater than or equal to 0.														
M_SAMPLE_INDEX(AIL_INT SampleIndex)	Specifies the index of a color-sample that has one color element. If the color-sample has multiple color elements, use M_SAMPLE_INDEX_ITEM() or M_SAMPLE_LABEL_ITEM().	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	SampleIndex Specifies an index value greater than or equal to 0.														
M_SAMPLE_INDEX_ITEM(AIL_INT SampleIndex, AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to a color-sample index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	SampleIndex Specifies the index of the color-sample that holds the color element (item). The index must be greater than or equal to 0.														
	ElementIndex Specifies the index of the color element (item). The index value must be greater than or equal to 0.														
M_SAMPLE_LABEL(AIL_INT SampleLabel)	Specifies the label of a color-sample that has one color element. If the color-sample has multiple color elements, use M_SAMPLE_INDEX_ITEM() or M_SAMPLE_LABEL_ITEM().	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	SampleLabel Specifies a label value greater than 0.														
M_SAMPLE_LABEL_ITEM(AIL_INT SampleLabel, AIL_INT ElementIndex)	Specifies the index of a color element (item), relative to a color-sample label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														

	SampleLabel Specifies the label of the color-sample that holds the color element (item). The label value must be greater than 0.															
	ElementIndex Specifies the index of the color element (item). The index value must be greater than or equal to 0.															
M_REFERENCE_SAMPLE	Specifies the reference color-sample. This value only applies to relative color calibration contexts (ContextId).	c		e	f	g	a		i		j	k	l	m	n	b

MaskBufferId

Specifies the identifier of the buffer to use to identify the masked (non-zero) pixels in the color element. This buffer must be a 1-band 8-bit unsigned image buffer.

If the mask is not the same size as the color element, the masking operation considers only the common region between both images. A common origin at the top-left corner of the mask image is assumed. If the mask is smaller than the color element, the color element region exceeding the mask is considered unmasked. If the mask is greater than the color element, the mask is clipped. An error is generated if the color-sample is entirely masked.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

To remove the mask, set the MaskBufferId parameter to **M_NULL**. An error is generated if you try removing a mask that does not exist.

MaskType

Specifies the type of mask to employ. Set this parameter to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of mask																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DONT_CARE	Specifies that Aurora Imaging Library ignores non-zero pixels in the color element's masked region. For example, masked pixels do not affect any of the resulting scores, when performing color matching. Masked pixels also do not affect the mapping Aurora Imaging Library establishes between each color-sample and the relevant color-sample, when transforming colors using relative color calibration.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolMatch

Synopsis

Match target areas with color-samples.

Syntax

```
void McolMatch(
    AIL_ID ContextId,                //in
    AIL_ID TargetColorImageId,       //in
    AIL_ID TargetColorProfileId,     //in
    AIL_ID AreaIdentifierImageId,    //in
    AIL_ID ColorResultOrDestImageId, //out
    AIL_INT64 ControlFlag            //in
)
```

Description

This function matches the entire target image, or specific areas of the target image, with the color-samples defined in a color matching context. This function allows you to identify a target area's best-matched color-sample, or to determine the proportion of color in a target area, based on the color-samples provided.

To match multiple target areas, you must specify an area identifier image. Each unique, non-zero label in the area identifier image defines an independent target area. Touching target areas with different labels are considered distinct. If you use an area identifier image that does not contain any non-zero labels, the match is not performed, no resulting image is produced, and all results regarding target areas and color-samples are unavailable.

To retrieve the results of the match, you must set the ControlFlag parameter to M_DEFAULT and specify a result buffer using the ColorResultOrDestImageId parameter; you must then use this result buffer with McolGetResult(). In this case, to draw specific features of a color-sample or of a color matching result, use McolDraw().

You can also use **McolMatch**() to retrieve a single image result, by setting the ControlFlag parameter to a drawing operation (M_DRAW_...) and specifying an image buffer with the ColorResultOrDestImageId parameter. In this case, since you do not have a result buffer, you will not be able to use McolGetResult().

By default, a minimum distance operation is used to match the color of the target area with the color of the color-sample. To change the default operation mode, use McolSetMethod(). You can also use McolSetMethod() to change the default distance type of the match operation.

For color-samples defined from an image, Aurora Imaging Library performs an estimation of the color. This estimate is based on color statistics, such as the mean; it is considered the color of the color-sample and is used in the match. The same kind of color estimation is also calculated for target areas, when matching with a minimum distance operation.

Aurora Imaging Library assumes the color you are using is in the context's source color space (McolAlloc()). Therefore all color data that you provide must be consistent. For example, if you allocate an RGB context, bands 0, 1, and 2 are assumed to be red, green, and blue; you will therefore not receive an error if you try to match RGB target areas with HSL color-samples. Similarly, you will not receive an error if you try to match a 16-bit target area with an 8-bit color-sample. Even when the color data is inconsistent, Aurora Imaging Library still processes it as is, which can produce misleading results.

You must set the color space encoding according to the actual dynamic range of your color data, using McolControl() with M_ENCODING. By default, the Color Analysis module assumes an 8-bit color space encoding

(regardless of the buffer depth).

Certain modifications to the color matching context, such as adding or deleting color-samples, or changing some control settings, require you to preprocess the color matching context (McolPreprocess()) before any subsequent call to **McolMatch**(). To inquire whether you should preprocess, use McolInquire() with M_PREPROCESSED.

The function has been optimized for performance under the following conditions:

- Buffers must be of the same type, as specified using McolControl() with M_ENCODING, preferably 8-bit or 16-bit.
- The target image buffer must have a width greater than or equal to 8 pixels.
- The area identifier image buffer must not be compressed (you cannot use MbufAllocColor() with M_COMPRESS).
- When using multiple samples, an 8-bit target image buffer, and McolSetMethod() with either M_EUCLIDEAN or M_MAHALANOBIS, you should set the maximum tolerance for the color distance to a value less than $2^{24/2}$, using McolControl() with M_DISTANCE_TOLERANCE.

  Note that $2^{24/2}$ is an internally established limit required for an optimal match process.
- When using multiple samples, a 16-bit target image buffer, and McolSetMethod() with either M_EUCLIDEAN or M_MAHALANOBIS, you should set the maximum tolerance for the color distance to a value less than $2^{53/2}$, using McolControl() with M_DISTANCE_TOLERANCE.

  Note that $2^{53/2}$ is an internally established limit required for an optimal match process.

Parameters

ContextId

Specifies the identifier of the color matching context. The color matching context must have been previously allocated on the required system using McolAlloc() with M_COLOR_MATCHING.

TargetColorImageId

Specifies the identifier of the target image buffer with which to match the color-samples.

Target images must be 3-band 8- or 16-bit unsigned images. The target image and the color-sample must have the same color space encoding.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

TargetColorProfileId

Specifies the identifier of the target image's reference color space. This is the standard used to interpret the target's color data when performing an internal conversion before the match operation, using McolSetMethod() with the CIELAB conversion mode.

This parameter must be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the target image's color space profile																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies to use standard RGB specifications (sRGB), as defined by the International Electrotechnical Commission (IEC) Project Team 61966-2-1.	c		e	f	g	a		i		j	k	l	m	n	b

AreaIdentifierImageId

Specifies the identifier of the image buffer containing the target area labels. Area identifier images can be unsigned 1-band, packed binary, 8- or 16-bit grayscale images.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

To match the entire target image as one target area, set this parameter to **M_NULL**.

ColorResultOrDestImageId

Specifies the identifier of the image buffer or result buffer in which to write the results of the color matching operation.

If you specify the identifier of an image buffer, it must be a 3-band buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

The bit depth of the image buffer must accommodate all the color-sample data drawn, including the background color, the outlier color, and the outlier label (when applicable).

The numerical value of the color-sample labels, the background pixels, and the outlier pixels and labels should be different so you can easily identify them if they are drawn (M_DRAW...). To modify these values, use McolControl() with M_SAMPLE_LABEL_VALUE, M_BACKGROUND_DRAW_COLOR, M_OUTLIER_DRAW_COLOR and M_OUTLIER_LABEL. Note that a pixel is considered background when it is outside the target areas, while outlier pixels are pixels inside a target area that did not match with any color-sample.

When the color of the color-sample is drawn, Aurora Imaging Library uses either its triplet value (M_TRIPLET) or its estimated value (for example, the average color) taken from a source image (M_IMAGE), depending on how it was defined (McolDefine()).

If you specify the identifier of a result buffer, it must have been previously allocated on the required system, using McolAllocResult().

ControlFlag

Specifies the results to produce.

For specifying that results should be written in the result buffer, this parameter must be set to the following value. In this case, you must set the ColorResultOrDestImageId parameter to the identifier of a result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that results should be written in the result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Calculates all possible results and writes them in the specified result buffer (ColorResultOrDestImageId).	c		e	f	g	a		i		j	k	l	m	n	b

For specifying that a specific image should be drawn in the image buffer, this parameter must be set to one the following values. In this case, you must set the ColorResultOrDestImageId parameter to the identifier of an image buffer. You can only draw one image at a time.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that a specific image result should be drawn in the image buffer																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_AREA_MATCH_USING_COLOR +	Draws, for each target area, the color of the best-matched color-sample. The background color and outlier color are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_DRAW_COLOR, respectively. For M_DRAW_AREA_MATCH_USING_COLOR, you must set the ColorResultOrDestImageId parameter to the identifier of a 3-band color image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_AREA_MATCH_USING_LABEL	Draws, for each target area, the label of the best-matched color-sample. The background color and outlier label are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_LABEL, respectively. For M_DRAW_AREA_MATCH_USING_LABEL, you must set the ColorResultOrDestImageId parameter to the identifier of a grayscale image buffer. To inquire the required buffer depth, use McolInquire() with M_SAMPLE_LABEL_SIZE_BIT.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_DISTANCE	Draws the distance between the estimated color (average color) of the target area (for an M_STAT_MIN_DIST operation mode) or the color of the target pixel (for an M_MIN_DIST_VOTE operation mode), and the color of its best-matched color-sample. Operation modes are set with McolSetMethod(). The background color and outlier color are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_DRAW_COLOR, respectively. For M_DRAW_DISTANCE, you must set the ColorResultOrDestImageId parameter to the identifier of a grayscale image buffer. To draw normalized distances, use McolControl() with M_DISTANCE_IMAGE_NORMALIZE before drawing. The outlier color, which is also drawn with M_DRAW_DISTANCE, can be difficult to identify if it is similar to the resulting distance. For example, if the outlier color is 0.0, you wouldn't be able to distinguish it if the resulting distance is also 0.0. In this case, you can use M_DRAW_..._USING_LABEL, which draws the outlier label. M_DRAW_DISTANCE is not available if the operation mode is set to M_HISTOGRAM_MATCHING or M_HISTOGRAM_VOTE (McolSetMethod()).	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_PIXEL_MATCH_USING_COLOR +	Draws, for each pixel in each target area, the color of the color-sample for which each pixel voted. The background color and outlier color are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_DRAW_COLOR, respectively. For M_DRAW_PIXEL_MATCH_USING_COLOR, you must set the ColorResultOrDestImageId parameter to the identifier of a 3-band color image buffer, and you must use McolSetMethod() with M_MIN_DIST_VOTE.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_PIXEL_MATCH_USING_LABEL	Draws, for each pixel in each target area, the label of the color-sample for which each pixel voted. The background color and outlier label are also drawn; to modify these values, use McolControl() with M_BACKGROUND_DRAW_COLOR and M_OUTLIER_LABEL, respectively. For M_DRAW_PIXEL_MATCH_USING_LABEL, you must use McolSetMethod() with M_MIN_DIST_VOTE, and you must set the ColorResultOrDestImageId parameter to the identifier of a grayscale image buffer. To inquire the required buffer depth, use McolInquire() with M_SAMPLE_LABEL_SIZE_BIT.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_DRAW_AREA_MATCH_USING_COLOR; M_DRAW_PIXEL_MATCH_USING_COLOR.

You can add the following value to the above-mentioned values to specify whether to invert colors.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to invert colors																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INVERTED_COLORS	Inverts the color of the best-matched color-sample in the specified image.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolPreprocess

Synopsis

Preprocess a relative color calibration or color matching context.

Syntax

```
void McolPreprocess(
    AIL_ID ContextId,     //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function prepares the specified relative color calibration or color matching context for an McolTransform() (relative color calibration) or McolMatch() (color matching) operation. Preprocessing calculates the necessary data for each color-sample contained in its respective context and specifies internal settings to optimize future calculations for speed and robustness. For a relative color calibration context, preprocessing also establishes the color mapping between each color-sample and the reference color-sample.

You must preprocess the relative color calibration or color matching context before the first call to McolTransform() or McolMatch(). Changes to the context, its settings, or one of its color-samples often require you to preprocess the context again. To inquire if this is the case, use McolInquire() with M_PREPROCESSED.

When you save the context, preprocessing changes are not saved. Upon restoration (McolRestore()), you must preprocess the context.

Parameters

ContextId

Specifies the identifier of the context to preprocess. The context must have been previously allocated on the required system using McolAlloc() with M_COLOR_CALIBRATION_RELATIVE or M_COLOR_MATCHING.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolProject

Synopsis

Perform a color projection.

Syntax

```
void McolProject(
    AIL_ID Src1ImageId,        //in
    AIL_ID Src2ImageOrArrayId, //in
    AIL_ID DestImageOrArrayId, //out
    AIL_ID DestMaskImageId,    //out
    AIL_INT64 Operation,       //in
    AIL_INT64 ControlFlag,     //in
    AIL_INT *StatusPtr         //out
)
```

Description

This function performs a color projection (transformation) of the source image, or returns a color projection matrix, which you can then apply to transform other images, using MimConvert(). **McolProject**() allows you to separate a selected color from a rejected one, or to convert color images to grayscale with minimal loss of information. You can also use this function to perform analytical tasks, such as calculating an image's covariance matrix or the principal components of an image's color information.

The calculated projection results are written in the specified destination buffer (DestImageOrArrayId). If the calculated projection results create an image, you can restrict the area to which results are written in the destination buffer using a mask image (DestMaskImageId).

Color projection is an operation that requires data defined in a Cartesian coordinate system (for example, RGB or CIELAB). A projection with HSL data would provide incoherent results.

> This function is optimized for images with a width (X-size) greater than seven pixels.

Parameters

Src1ImageId

Specifies the identifier of the source image buffer on which to perform the projection transformation. This must be a 3-band 8- or 16-bit unsigned image. Source images are interpreted per band, independently of the Aurora Imaging Library buffer format.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

When separating colors (M_COLOR_SEPARATION) and using an Aurora Imaging Library array buffer to specify the separation colors (Src2ImageOrArrayId), set this parameter to **M_NULL**. In this case, the destination buffer will contain the resulting projection matrix, rather than the transformed image.

Src2ImageOrArrayId

Specifies the identifier of an image buffer or Aurora Imaging Library array buffer that controls how the source pixels are interpreted.

See the Parameter associations section for possible values that can be specified.

DestImageOrArrayId

Specifies the identifier of the destination buffer in which to write the color projection (transformation) results. The destination buffer can be either an image or an Aurora Imaging Library array buffer.

Note that when the result is a matrix, you must set this parameter to an Aurora Imaging Library array buffer, allocated using MbufAlloc...() with M_ARRAY.

See the Parameter associations section for possible values that can be returned.

DestMaskImageId

Specifies the identifier of the destination mask image. The mask image must be monochrome. You can only specify a mask if the destination buffer is an image. If you do not require a mask, set this parameter to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

The destination mask determines where to write the result of the projection operation. Only masked pixels (non-zero) will be written to, in the destination image. The destination mask does not affect calculations.

Operation

Specifies the color projection operation.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies the dynamic range on which the resulting projection is mapped. This is only applicable for M_PRINCIPAL_COMPONENT_PROJECTION.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

StatusPtr

Specifies the address of the variable in which to write the status of the color projection operation. If you do not require this information, set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **Src2ImageOrArrayId**, **Operation**, and **ControlFlag** parameters and possible values returned to the **DestImageOrArrayId** and **StatusPtr** parameters.

- For implementing the color projection

Note that you should set any unused parameters to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For implementing the color projection																	
Operation		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	Src2ImageOrArrayId																

	DestImageOrArrayId - Possible values returned															
	ControlFlag															
	StatusPtr - Possible values returned															
M_COLOR_SEPARATION		Removes a color from the source image. In this case, you must identify the background, selected, and rejected colors. The rejected color indicates the color to remove.														
	Src2ImageOrArrayId	Specifies the identifier of the buffer identifying the background, selected, and rejected pixel colors with which to perform the color separation. If Src1ImageId is set to **M_NULL**, you must set Src2ImageOrArrayId to an Aurora Imaging Library array buffer. In this case, the destination buffer will contain the resulting projection matrix for the specified background, selected, and rejected pixels colors, rather than the transformed source image.	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer identifier	Specifies the identifier of a 3x3 Aurora Imaging Library array buffer, where each row defines, in the following order, the background, the selected, and the rejected colors, to use to compute the projection matrix.	c		e	f	g	a		i		j	k	l	m	n	b

	Image buffer identifier	Specifies the identifier of a data identification image buffer. This image determines which areas of the source image (Src1ImageId) have the background, selected, and rejected colors; these colors are then used to calculate the projection matrix. To identify the source image colors, you must set the corresponding pixels in the data identification image to one of the label values below. **M_BACKGROUND_LABEL** — Identifies a background-color pixel. **M_REJECTED_LABEL** — Identifies a rejected-color pixel. **M_SELECTED_LABEL** — Identifies a selected-color pixel. The calculation uses the average color value of all the pixels in the source image identified as background, selected, and rejected. You must assign at least one pixel in the data identification image to each of these values. Pixels in the source image that do not correspond to one of these values are ignored in the calculation. If the data identification image is smaller than the source image, the outlying source image pixels are ignored. The data identification buffer must be a 1-band 8-bit unsigned image buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
	DestImageOrArrayId	Specifies the identifier of the destination buffer in which to write the color separation results. If Src1ImageId is set to **M_NULL**, you must set DestImageOrArrayId to an Aurora Imaging Library array buffer.	c		e	f	g	a		i		j	k	l	m	n	b

		Array buffer identifier	Specifies the identifier of a 4x3 Aurora Imaging Library array buffer, of type *float*, to store the resulting projection matrix. You can use this matrix with MimConvert() to separate colors in any 3-band color image. Be certain that the color distribution of all source images that use this matrix is similar, otherwise unpredictable results can occur.	c		e	f	g	a		i		j	k	l	m	n	b
		Image buffer identifier	Specifies the identifier of a 3-band color image buffer, to store the transformed (color separated) source image. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
	StatusPtr		Returns the resulting status of the M_COLOR_SEPARATION operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_3_COLORS_COLLINEAR	Specifies an unsatisfactory status, indicating that the selected, rejected, and background colors are collinear. If DestImageOrArrayId specifies an image buffer, it is a copy of the source image. If DestImageOrArrayId specifies an array buffer, it is filled with the identity matrix such that, using this array with MimConvert() copies the source image into the destination image.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NO_BACKGROUND_DEFINED	Specifies an unsatisfactory status, indicating that no background color was defined with the Src2ImageOrArrayId parameter. If DestImageOrArrayId specifies an image buffer, it is a copy of the source image. If DestImageOrArrayId specifies an array buffer, it is filled with the identity matrix such that, using this array with MimConvert() copies the source image into the destination image.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NO_REJECTED_DEFINED	Specifies an unsatisfactory status, indicating that no rejected color was defined with the Src2ImageOrArrayId parameter. If DestImageOrArrayId specifies an image buffer, it is a copy of the source image. If DestImageOrArrayId specifies an array buffer, it is filled with the identity matrix such that, using this array with MimConvert() copies the source image into the destination image.	c		e	f	g	a		i		j	k	l	m	n	b

	M_NO_SELECTED_DEFINED	Specifies an unsatisfactory status, indicating that no selected color was defined with the Src2ImageOrArrayId parameter. If DestImageOrArrayId specifies an image buffer, it is a copy of the source image. If DestImageOrArrayId specifies an array buffer, it is filled with the identity matrix such that, using this array with MimConvert() copies the source image into the destination image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REJECTED_EQUAL_BACKGROUND	Specifies an unsatisfactory status, indicating that the rejected color equals the background color. If DestImageOrArrayId specifies an image buffer, it is a copy of the source image. If DestImageOrArrayId specifies an array buffer, it is filled with the identity matrix such that, using this array with MimConvert() copies the source image into the destination image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REJECTED_EQUAL_SELECTED	Specifies an unsatisfactory status, indicating that the rejected color equals the selected color. If DestImageOrArrayId specifies an image buffer, it is a copy of the source image. If DestImageOrArrayId specifies an array buffer, it is filled with the identity matrix such that, using this array with MimConvert() copies the source image into the destination image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SELECTED_EQUAL_BACKGROUND	Specifies an unsatisfactory status, indicating that the selected color equals the background color. In this case, although the operation was performed, Aurora Imaging Library internally used, as the selected color, a color that is perpendicular to the rejected color.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SUCCESS	Specifies a successful operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_COVARIANCE		Calculates the covariance matrix of the source image's color information. This is a measure of the color variation.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2ImageOrArrayId	Specifies the identifier of the buffer that controls how the source pixels are interpreted when performing the covariance calculation.	c		e	f	g	a		i		j	k	l	m	n	b

	M_NULL	Specifies to use the entire source image to compute the covariance matrix.	c		e	f	g	a		i		j	k	l	m	n	b
	Image buffer identifier	Specifies the identifier of a data identification image buffer. This image determines which areas of the source image (Src1ImageId) to use to compute the covariance matrix. To specify an area, you must set the corresponding pixels in the data identification image to **M_SOURCE_LABEL**. You must assign at least one pixel in the data identification image to **M_SOURCE_LABEL**. Pixels in the source image that do not correspond to **M_SOURCE_LABEL** are ignored in the calculation. If the data identification image is smaller than the source image, the outlying source image pixels are ignored. The data identification buffer must be a 1-band 8-bit unsigned image buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
DestImageOrArrayId		Specifies the identifier of a 3x3 or 4x3 Aurora Imaging Library array buffer, of type *float*, to store the resulting covariance matrix. The data returned depends on the array's dimensions: • 3x3. The covariance matrix elements are returned columnwise. $\begin{bmatrix} var(Band0) & cov(Band0, Band1) & cov(Band0, Band2) \\ cov(Band1, Band0) & var(Band1) & cov(Band1, Band2) \\ cov(Band2, Band0) & cov(Band2, Band1) & var(Band2) \end{bmatrix}$ *var* = variance, *cov* = covariance; covariance matrix is symmetrical, so: *CovMatrix(I, j) = CovMatrix(j, I)* • 4x3. In addition to the 3x3 array results, the mean color of the source image areas (**M_SOURCE_LABEL**) is returned in the fourth column. $\begin{bmatrix} var(Band0) & cov(Band0, Band1) & cov(Band0, Band2) & Band0MeanValue \\ cov(Band1, Band0) & var(Band1) & cov(Band1, Band2) & Band1MeanValue \\ cov(Band2, Band0) & cov(Band2, Band1) & var(Band2) & Band2MeanValue \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
StatusPtr		Returns the resulting status of the M_COVARIANCE operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_SOURCE_DEFINED	Specifies an unsatisfactory status, indicating that there was no pixel in the data identification image that was set to **M_SOURCE_LABEL**. The resulting matrix (DestImageOrArrayId) is filled with zeros.	c		e	f	g	a		i		j	k	l	m	n	b

		M_SUCCESS	Specifies a successful operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_COMPONENT_PROJECTION			Performs a principal component projection of the source image. This converts each color pixel to a grayscale value with a point-to-point projection of the source image, onto its first principal component, using a grayscale LUT. The projection is a mathematically optimal color-to-grayscale transformation, which uses the distribution of the source image's color data to calculate the best grayscale conversion possible, minimizing the loss of information. Aurora Imaging Library performs a principal component analysis (PCA) to calculate the color image's principal components. The strongest is the first principal component. If no principal component is the strongest, one is arbitrarily considered the first principal component.	c		e	f	g	a		i		j	k	l	m	n	b
	Src2ImageOrArrayId		Specifies the identifier of the buffer that controls how the source pixels are interpreted to calculate the first principal component, which is used to perform the principal component projection.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NULL	Specifies to use the entire source image to perform the principal component projection.	c		e	f	g	a		i		j	k	l	m	n	b

		Image buffer identifier	Specifies the identifier of a data identification image buffer. This image determines which areas of the source image (Src1ImageId) to use to calculate the first principal component, which is used to perform the principal component projection. To specify an area, you must set the corresponding pixels in the data identification image to **M_SOURCE_LABEL**. If there is ambiguity between which colors should be designated as light and dark, set some pixels in the data identification image to **M_BRIGHT_LABEL** and **M_DARK_LABEL**. **M_BRIGHT_LABEL** identifies source pixels that represent the color to project on the brightest side of the grayscale palette, while **M_DARK_LABEL** identifies source pixels that represent the color to project on the darkest side. These labels are optional and pixels designated as bright or dark are not used in the principal component projection for anything other than determining which colors are light and dark. Therefore, use as few pixels as necessary for this purpose. Aurora Imaging Library uses the mean color of the bright or dark pixels if you set multiple pixels to bright or dark. You must assign at least one pixel in the data identification image to **M_SOURCE_LABEL**. Pixels in the source image that do not correspond to **M_SOURCE_LABEL** are ignored in the calculation. If the data identification image is smaller than the source image, the outlying source image pixels are ignored. The data identification image must be a 1-band 8-bit unsigned image buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
	DestImageOrArrayId		Specifies the identifier of the destination buffer in which to write the principal component results.	c		e	f	g	a		i		j	k	l	m	n	b
		Array buffer identifier	Specifies the identifier of a 4x1 Aurora Imaging Library array buffer, of type *float*, to store the resulting projection matrix. You can use this matrix with MimConvert() to perform a color-to-grayscale transformation of any 3-band color image. Be certain that the color distribution of all source images that use this matrix is similar, otherwise unpredictable results can occur.	c		e	f	g	a		i		j	k	l	m	n	b

	Image buffer identifier	Specifies the identifier of a 1-band image buffer, to store the transformed source image. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
ControlFlag		Specifies whether Aurora Imaging Library should perform the projection according to the dynamic range (minimum and maximum pixel values) established from all the pixels in the source image, or according to the dynamic range established from only the specified source image pixels (**M_SOURCE_LABEL**) identified with the data identification image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Performs the projection according to the dynamic range of all the pixels in the source image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MASK_CONTRAST_ENHANCEMENT	Performs the projection according to the dynamic range of the pixels identified with the data identification image. The result is similar to increasing the contrast. When using M_MASK_CONTRAST_ENHANCEMENT, Aurora Imaging Library saturates the projected unidentified source image colors that are outside the dynamic range of the identified source image pixels' color; that is, Aurora Imaging Library maps those colors to either the minimum or maximum allowable grayscale value. In this case, the projection result can contain sections that have lost a certain degree of contrast information.	c		e	f	g	a		i		j	k	l	m	n	b
StatusPtr		Returns the resulting status of the M_PRINCIPAL_COMPONENT_PROJECTION operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NO_SOURCE_DEFINED	Specifies an unsatisfactory status, indicating that there was no pixel in the data identification image that was set to **M_SOURCE_LABEL**. If DestImageOrArrayId specifies an image buffer, it is a copy of the source image. If DestImageOrArrayId specifies an array buffer, it is filled with the identity matrix such that, using this array with MimConvert() copies the source image into the destination image.	c		e	f	g	a		i		j	k	l	m	n	b

	M_SUCCESS	Specifies a successful operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSTABLE_POLARITY	Specifies an unsatisfactory status, indicating that the polarity of the projection is unstable. In this case, although the operation was performed, it was not possible to confidently determine, in a consistent and repeatable way, which pixels should be projected to the brightest/darkest side of the grayscale palette.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRINCIPAL_COMPONENTS		Calculates the principal components of the source image's color information. The principal components correspond to the three eigenvectors of the image's covariance matrix. With M_PRINCIPAL_COMPONENTS, you can also retrieve each eigenvector's eigenvalue and the source image's average color. To calculate the principal components, Aurora Imaging Library performs a principal component analysis (PCA).	c		e	f	g	a		i		j	k	l	m	n	b
Src2ImageOrArrayId		Specifies the identifier of the buffer that controls how the source pixels are interpreted to perform the principal component calculation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies to use the entire source image to calculate the principal components.	c		e	f	g	a		i		j	k	l	m	n	b
	Image buffer identifier	Specifies the identifier of a data identification image buffer. This image determines which areas of the source image (Src1ImageId) to use to calculate the principal components. To specify an area, you must set the appropriate pixels in the data identification image to **M_SOURCE_LABEL**. You must assign at least one pixel in the data identification image to **M_SOURCE_LABEL**. Pixels in the source image that do not correspond to **M_SOURCE_LABEL** are ignored in the calculation. If the data identification image is smaller than the source image, the outlying source image pixels are ignored. The data identification image must be a 1-band 8-bit unsigned image buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

DestImageOrArrayId	Specifies the identifier of the destination buffer in which to write the resulting data for each of the three principal components.	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of a 3x3, 4x3, or 5x3 Aurora Imaging Library array buffer, of type *float*, to store the resulting data. The data returned depends on the array's dimensions: • 3x3. The three eigenvectors are returned columnwise in decreasing order of strength. • 4x3. In addition to the 3x3 array results, each eigenvectors's eigenvalue is returned in the fourth column. The eigenvalue corresponds to the eigenvector's strength. • 5x3. In addition to the 4x3 array results, the mean color of the source image areas (**M_SOURCE_LABEL**) is returned in the fifth column, band-by-band: *SourceMeanValueBand0*, *SourceMeanValueBand1*, and *SourceMeanValueBand2*.	c		e	f	g	a		i		j	k	l	m	n	b
StatusPtr	Returns the resulting status of the M_PRINCIPAL_COMPONENTS operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_SOURCE_DEFINED	Specifies an unsatisfactory status, indicating that there was no pixel in the data identification image that was set to **M_SOURCE_LABEL**. The resulting matrix (DestImageOrArrayId) is filled with zeros.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUCCESS	Specifies a successful operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSTABLE_POLARITY	Specifies an unsatisfactory status, indicating that the direction of the first principal component is unstable. In this case, although the operation was performed, it was not possible to align the first principal component, in a consistent and repeatable way, with the gray axis (from black to white).	c		e	f	g	a		i		j	k	l	m	n	b

		M_UNSTABLE_PRINCIPAL_COMPONENT_2	Specifies an unsatisfactory status, indicating that the direction of the third principal component (least important) is unstable. Although the operation was performed, this status result typically means that there were small variances along the color component's direction.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UNSTABLE_PRINCIPAL_COMPONENTS_12	Specifies an unsatisfactory status, indicating that the direction of the second and third principal components are unstable. Although the operation was performed, this status result typically means that there were small variances along the color component's direction.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UNSTABLE_PRINCIPAL_COMPONENTS_012	Specifies an unsatisfactory status, indicating that the direction of all principal components are unstable. The resulting matrix (DestImageOrArrayId) is filled with zeros.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolRestore

Synopsis

Restore a color matching or relative color calibration context from disk.

Syntax

```
AIL_ID McolRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextIdPtr         //out
)
```

Description

This function restores a color matching or a relative color calibration context that was previously saved to a file, using McolSave() or McolStream(). This function restores all of the context's settings (and color-samples) that were in effect when the context was saved. A restored context is not preprocessed; therefore, you must call McolPreprocess() before calling McolMatch() (color matching) or McolTransform() (relative color calibration).

When the restored color matching or relative color calibration is no longer required, release it using McolFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the context. The function handles (internally) the opening and closing of the file.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). For easier use with other Aurora Imaging software products, files for color matching contexts should use the MCOL file extension. Files for relative color calibration contexts should use the MCCC file extension. To specify the file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextIdPtr

Specifies the address of the variable in which to write the color matching context or relative calibration context identifier or specifies the data type that the function should use to return the context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated color matching or relative color calibration context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated color matching or relative color calibration context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_COL_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the color matching or relative color calibration context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the color matching context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored color matching context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the relative color calibration context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored relative color calibration context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the color matching context or relative color calibration context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_COL_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolSave

Synopsis

Save a color matching or relative color calibration context to a file.

Syntax

```
void McolSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextId,            //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves to disk all the information about the previously allocated color matching or relative color calibration context (and the color-samples contained therein). However, preprocessing information is not saved. You can reload the saved information, using McolRestore() or McolStream().

Parameters

FileName

Specifies the name and path of the file in which to save the context. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). For easier use with other Aurora Imaging software products, when saving a color matching context to a file, use the MCOL file extension. When saving a relative color calibration context to a file, use the MCCC file extension. To save the file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextId

Specifies the identifier of the context to save. The context must have been previously allocated on the required system using McolAlloc() with M_COLOR_CALIBRATION_RELATIVE or M_COLOR_MATCHING.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolSetMethod

Synopsis

Set the color matching or relative color calibration strategy.

Syntax

```
void McolSetMethod(
    AIL_ID ContextId,                       //in
    AIL_INT64 OperationMode,                //in
    AIL_INT64 DistTypeOrCalIntent,          //in
    AIL_INT64 ConversionModeOrComputeOption, //in
    AIL_INT64 ControlFlag                   //in
)
```

Description

This function sets the strategy with which to perform color matching or relative color calibration, when calling McolMatch() or McolTransform(), respectively. The strategy is based on the operation and either the color distance (for color matching) or the color calibration intent (for relative color calibration). By manipulating these settings, you can affect the resulting color-samples that Aurora Imaging Library matches to the target areas, or the color-sample mappings that Aurora Imaging Library uses to transform color data with relative color calibration.

If you perform color matching using an RGB source color space, you can use this function to convert your RGB color data to a different color space before the match operation. To do so, explicitly specify a conversion mode (ConversionModeOrComputeOption).

The strategy set with this function is part of the specified context. For this function's settings to take effect, you must preprocess the context (McolPreprocess()).

Parameters

ContextId

Specifies the identifier of the color matching or relative color calibration context. The context must have been previously allocated on the required system using McolAlloc() with M_COLOR_MATCHING or M_COLOR_CALIBRATION_RELATIVE.

See the Parameter associations section for possible values that can be specified.

OperationMode

Specifies the operation.

For specifying the operation with which to perform McolMatch(), set this parameter to one of the following values. The ContextId parameter must indicate the identifier of a color matching context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the distance operation (color matching) INQ																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_STAT_MIN_DIST.	c		e	f	g	a		i		j	k	l	m	n	b

M_HISTOGRAM_MATCHING	Specifies a minimum distance operation, based on color histograms. When using M_HISTOGRAM_MATCHING, you must set M_DISTANCE_TOLERANCE_MODE to M_ABSOLUTE. Color histograms are computed for each target area and defined color-sample, and a match score between each target area histogram and the color-sample histograms is generated. The color-sample with the highest score above the acceptance (McolControl() with M_ACCEPTANCE) is the target area's best-matched color-sample. Note that you cannot use M_HISTOGRAM_MATCHING if the distance type is M_MAHALANOBIS.	c		e	f	g	a		i		j	k	l	m	n	b
M_HISTOGRAM_VOTE	Specifies a match operation based on histogram voting. Aurora Imaging Library computes color histograms for each defined color-sample and, for each target pixel, identifies its histogram bin. Each target pixel votes for the best color-sample, provided its histogram bin is not empty. The number of votes that a color-sample accumulates determines its score. The color-sample with the highest score above the acceptance (McolControl() with M_ACCEPTANCE) is the target area's best-matched color-sample. In the case of a tie, where more than one color-sample receives the same highest number of votes, the color-sample with the highest label value is chosen as the best-match. With histogram voting, you must set the distance type to M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_DIST_VOTE	Specifies a minimum distance operation, based on pixel voting. Color statistics are calculated (typically the mean) for each color-sample defined in the context, and the color distance between each pixel in each target area and each color-sample is determined. Each target pixel then votes for the closest color-sample, which is also within the distance tolerance (McolControl() with M_DISTANCE_TOLERANCE). The number of votes that a color-sample accumulates determines its score. The color-sample with the highest score above the acceptance (McolControl() with M_ACCEPTANCE) is the target area's best-matched color-sample. In the case of a tie, where more than one color-sample receives the same highest number of votes, the color-sample with the highest label value is chosen as the best-match.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_MIN_DIST	Specifies a minimum distance operation, based on pixel statistics. Color statistics are calculated (typically the mean) for each target area and for each color-sample defined in the context, and the color distance between each target area and each color-sample is determined. The resulting distances determine the score of the color-samples. If a distance is not within a color-sample's distance tolerance, this color-sample's score is 0%. The color-sample with the highest score above the acceptance (McolControl() with M_ACCEPTANCE) is the target area's best-matched color-sample.	c		e	f	g	a		i		j	k	l	m	n	b

For specifying the operation with which to establish the mapping that Aurora Imaging Library associates between each color-sample and the reference color-sample, set this parameter to one of the following values. The ContextId parameter must indicate the identifier of a relative color calibration context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the mapping operation (relative color calibration) INQ																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_HISTOGRAM_BASED.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_TO_COLOR	Specifies that a color-sample's mapping is based on transforming its color to the color of the reference color-sample on a point-to-point basis. It is recommended to have a pixel-wise correspondence between the color-sample and the reference color-sample. M_COLOR_TO_COLOR is the most precise operation, and the least flexible. Minor color variations and outlier color data can have a major impact. Use this operation when the conditions of your application are stable and you require an exact reproduction of color.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_MEAN_VARIANCE	Specifies that a color-sample's mapping is based on transforming its mean and variance (standard deviation) to resemble the mean and variance of the reference color-sample. This is the fastest operation, and the least precise (most general). Use it when the color distribution of the reference color-sample and the color-samples is dissimilar, when conditions are not stable, or when other operations fail. M_GLOBAL_MEAN_VARIANCE can avoid the introduction of false colors if the acquisition scene (image and color data) varies dramatically between the reference color-sample and the color-samples.	c		e	f	g	a		i		j	k	l	m	n	b

		c		e	f	g	a		i		j	k	l	m	n	b
M_HISTOGRAM_BASED	Specifies that a color-sample's mapping is based on using enhanced color distribution information to transform its color data to better resemble the color data of the reference color-sample. Use this operation when the color distribution between the reference color-sample and the color-samples is similar. M_HISTOGRAM_BASED is a compromise between point-to-point (high precision, low flexibility) and global mean variance (low precision, high flexibility) operations. It offers good precision without requiring a point-to-point correspondence. It also offers good flexibility, provided that colors are somewhat comparable. This operation is typically the longest to process.															

DistTypeOrCalIntent INQ

Specifies the type of distance (for color matching) or color calibration intent (for relative color calibration) that the operation uses. Distance is the difference in color between the color-sample and the target area. Except for the M_MAHALANOBIS distance type, which uses the covariance of the color-sample, Aurora Imaging Library uses the mean of the colors to calculate the distance. Color calibration intent is the extent to which the color data of the color-samples must resemble the color data of the reference color-sample.

See the Parameter associations section for possible values that can be specified.

ConversionModeOrComputeOption INQ

Specifies the conversion mode (for color matching) or compute option (for relative color calibration) that the operation uses. Conversion mode refers to the color space to which the RGB source color data will be converted before performing McolMatch(). You can only convert RGB source color data to CIELAB or HSL when matching colors. By default, this type of conversion is not performed (or allowed), and you should set this parameter to **M_DEFAULT** (same as M_NONE). Compute option indicates whether the operation uses all pixel values or color statistics.

When specifying a conversion mode, the match operation uses the converted color values. For example, if you are using the ConversionModeOrComputeOption parameter to convert your RGB color data to HSL, the match will be performed with the HSL version of the color.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The tables below list possible values for the **ContextId**, **DistTypeOrCalIntent**, and **ConversionModeOrComputeOption** parameters.

- For specifying the type of distance calculation (color matching)
- For specifying the color calibration intent (quality) of the mapping operation (relative color calibration)

The following ContextId, DistTypeOrCalIntent, and ConversionModeOrComputeOption parameter settings specify the type of distance calculation to use when calling McolMatch(). The ContextId parameter must indicate the identifier of a color matching context.

Except for converting RGB source color data to CIELAB or HSL when matching colors, the ConversionModeOrComputeOption parameter is unused and should be set to **M_DEFAULT** (same as M_NONE).

The settings in the following table are restricted according to the color matching context's source color space, as set using McolAlloc() with M_RGB, M_HSL, or M_CIELAB. Note that the only distance type available for an M_HSL source color space, as indicated in the table below, is M_MANHATTAN (which is also the default for HSL).

The M_CMC... and M_CIE... settings are recommended for advanced users and are typically applied to improve the Delta-E performance when dealing with minor color variances in industrial color difference evaluation. These distance types follow the standards of the International Commission on Illumination (CIE), as established in their technical report on Colorimetry (CIE 15:2004).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of distance calculation (color matching)																	
ContextId DistTypeOrCallIntent ConversionModeOrComputeOption	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
CIELAB color matching context ID	Specifies a color matching context that is using an M_CIELAB source color space.															
M_DEFAULT	Specifies the default distance type. The default is the same as M_EUCLIDEAN.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIE94_GRAPHIC_ARTS	Specifies to use the CIE94 color distance with weighting factors pertaining to the graphic arts industry, as established by the CIE. The weighting factors are: *KL* = 1, *K1* = 0.045, and *K2* = 0.015. For more information, including the relevant formulas, refer to the technical report on Colorimetry (CIE 15:2004). This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b

	M_CIE94_TEXTILE	Specifies to use the CIE94 color distance with weighting factors pertaining to the textile industry, as established by the CIE. The weighting factors are: *KL* = 2, *K1* = 0.048, *K2* = 0.014. For more information, including the relevant formulas, refer to the technical report on Colorimetry (CIE 15:2004). This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
	M_CIEDE2000	Specifies to use the CIEDE2000 color distance, as established by the CIE. This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
	M_CMC_ACCEPTABILITY	Specifies to use the CMC(*l*: *c*) color distance that has been adjusted for acceptability evaluation, as established by the CIE. The adjusted parameters are: lightness (*l*) = 2 and chroma (*c*) = 1. For more information, including the relevant formulas, refer to the technical report on Colorimetry (CIE 15:2004). This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
	M_CMC_PERCEPTIBILITY	Specifies to use the CMC(*l*: *c*) color distance that has been adjusted for perceptibility evaluation, as established by the CIE. The adjusted parameters are: lightness (*l*) = 1 and chroma (*c*) = 1. For more information, including the relevant formulas, refer to the technical report on Colorimetry (CIE 15:2004). This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b

	M_DELTA_E	Specifies to use a Delta-E color distance, as defined by the International Commission on Illumination (CIE). A Delta-E color distance is the same as a Euclidean color distance (M_EUCLIDEAN), but specialized for CIELAB.	c		e	f	g	a		i		j	k	l	m	n	b
	M_EUCLIDEAN	Specifies to use a Euclidean color distance. A Euclidean distance is the square root of the sum of the squared differences between the color of the color-sample and the color of the target area.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAHALANOBIS	Specifies to use a Mahalanobis color distance. A Mahalanobis distance is calculated between the color of the target area and, typically, the covariance of the color-sample. This distance, between a color and a distribution of colors, is similar to a Euclidean distance between the mean of the two colors, but weighted by the inverse of the covariance of the distribution. This implies that the more a color distribution varies in a direction within the color space, the less important is the distance in that direction. Since the covariance matrix of the color-sample is used, the color-sample should usually come from a an image, and not be defined as a triplet (McolDefine() with M_IMAGE instead of M_TRIPLET). However, if you provide a color constant (triplet) as the second source, Mahalanobis behaves very much like Euclidean and will yield similar results. M_MAHALANOBIS is not available if the operation mode is M_HISTOGRAM_MATCHING.	c		e	f	g	a		i		j	k	l	m	n	b

	M_MANHATTAN	Specifies to use a Manhattan color distance. A Manhattan distance is the sum of the absolute value of the differences between the color of the color-sample and the color of the target area. For an HSL color space, the distance between angular coordinates is equal to the smallest angular difference, rather than the absolute value of the difference.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE	Specifies that no distance calculations are used. Instead, pixels in the target image vote for histogram bins. The histogram bins each have a range of values, and each pixel in the target image votes for the histogram bin whose range includes its value. M_NONE can only be set with M_HISTOGRAM_VOTE, and vice versa.	c		e	f	g	a		i		j	k	l	m	n	b
HSL color matching context ID		Specifies a color matching context that is using an M_HSL source color space.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default distance type. The default is the same as M_MANHATTAN.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MANHATTAN	Specifies to use a Manhattan color distance. A Manhattan distance is the sum of the absolute value of the differences between the color of the color-sample and the color of the target area. For an HSL color space, the distance between angular coordinates is equal to the smallest angular difference, rather than the absolute value of the difference.	c		e	f	g	a		i		j	k	l	m	n	b

	M_NONE	Specifies that no distance calculations are used. Instead, pixels in the target image vote for histogram bins. The histogram bins each have a range of values, and each pixel in the target image votes for the histogram bin whose range includes its value. M_NONE can only be set with M_HISTOGRAM_VOTE, and vice versa.	c		e	f	g	a		i		j	k	l	m	n	b
RGB color matching context ID		Specifies a color matching context that is using an M_RGB source color space.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default distance type. The default is the same as M_EUCLIDEAN.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation. When converting RGB source colors before the match using **McolSetMethod**() with the CIELAB conversion mode, Aurora Imaging Library requires uncorrected (linear) color data. Therefore, if gamma correction has been applied on the RGB source color data, you must remove it using McolControl() with M_CONVERSION_GAMMA set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_HSL	Specifies to internally convert RGB colors to HSL before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NONE	Specifies no internal conversion.	c		e	f	g	a		i		j	k	l	m	n	b

M_CIE94_GRAPHIC_ARTS	Specifies to use the CIE94 color distance with weighting factors pertaining to the graphic arts industry, as established by the CIE. The weighting factors are: *KL* = 1, *K1* = 0.045, and *K2* = 0.015. For more information, including the relevant formulas, refer to the technical report on Colorimetry (CIE 15:2004). This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIE94_TEXTILE	Specifies to use the CIE94 color distance with weighting factors pertaining to the textile industry, as established by the CIE. The weighting factors are: *KL* = 2, *K1* = 0.048, *K2* = 0.014. For more information, including the relevant formulas, refer to the technical report on Colorimetry (CIE 15:2004). This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIEDE2000	Specifies to use the CIEDE2000 color distance, as established by the CIE. This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_CMC_ACCEPTABILITY	Specifies to use the CMC(*l*: *c*) color distance that has been adjusted for acceptability evaluation, as established by the CIE. The adjusted parameters are: lightness (*l*) = 2 and chroma (*c*) = 1. For more information, including the relevant formulas, refer to the technical report on Colorimetry (CIE 15:2004). This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_CMC_PERCEPTIBILITY	Specifies to use the CMC(*l*: *c*) color distance that has been adjusted for perceptibility evaluation, as established by the CIE. The adjusted parameters are: lightness (*l*) = 1 and chroma (*c*) = 1. For more information, including the relevant formulas, refer to the technical report on Colorimetry (CIE 15:2004). This distance type does not support band selection; that is, McolControl() with M_BAND_MODE must be set to M_ALL_BANDS (or M_DEFAULT).	c		e	f	g	a		i		j	k	l	m	n	b
M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DELTA_E	Specifies to use a Delta-E color distance, as defined by the International Commission on Illumination (CIE). A Delta-E color distance is the same as a Euclidean color distance (M_EUCLIDEAN), but specialized for CIELAB.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b

	M_EUCLIDEAN	Specifies to use a Euclidean color distance. A Euclidean distance is the square root of the sum of the squared differences between the color of the color-sample and the color of the target area.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation. When converting RGB source colors before the match using **McolSetMethod**() with the CIELAB conversion mode, Aurora Imaging Library requires uncorrected (linear) color data. Therefore, if gamma correction has been applied on the RGB source color data, you must remove it using McolControl() with M_CONVERSION_GAMMA set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NONE	Specifies no internal conversion.	c		e	f	g	a		i		j	k	l	m	n	b

M_MAHALANOBIS	Specifies to use a Mahalanobis color distance. A Mahalanobis distance is calculated between the color of the target area and, typically, the covariance of the color-sample. This distance, between a color and a distribution of colors, is similar to a Euclidean distance between the mean of the two colors, but weighted by the inverse of the covariance of the distribution. This implies that the more a color distribution varies in a direction within the color space, the less important is the distance in that direction. Since the covariance matrix of the color-sample is used, the color-sample should usually come from a an image, and not be defined as a triplet (McolDefine() with M_IMAGE instead of M_TRIPLET). However, if you provide a color constant (triplet) as the second source, Mahalanobis behaves very much like Euclidean and will yield similar results. M_MAHALANOBIS is not available if the operation mode is M_HISTOGRAM_MATCHING.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies no internal conversion.	c		e	f	g	a		i		j	k	l	m	n	b
M_MANHATTAN	Specifies to use a Manhattan color distance. A Manhattan distance is the sum of the absolute value of the differences between the color of the color-sample and the color of the target area. For an HSL color space, the distance between angular coordinates is equal to the smallest angular difference, rather than the absolute value of the difference.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b

		M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation. When converting RGB source colors before the match using **McolSetMethod**() with the CIELAB conversion mode, Aurora Imaging Library requires uncorrected (linear) color data. Therefore, if gamma correction has been applied on the RGB source color data, you must remove it using McolControl() with M_CONVERSION_GAMMA set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_HSL	Specifies to internally convert RGB colors to HSL before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NONE	Specifies no internal conversion.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NONE		Specifies that no distance calculations are used. Instead, pixels in the target image vote for histogram bins. The histogram bins each have a range of values, and each pixel in the target image votes for the histogram bin whose range includes its value. M_NONE can only be set with M_HISTOGRAM_VOTE, and vice versa.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_NONE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_CIELAB	Specifies to internally convert RGB colors to CIELAB before the match operation. When converting RGB source colors before the match using **McolSetMethod**() with the CIELAB conversion mode, Aurora Imaging Library requires uncorrected (linear) color data. Therefore, if gamma correction has been applied on the RGB source color data, you must remove it using McolControl() with M_CONVERSION_GAMMA set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_HSL	Specifies to internally convert RGB colors to HSL before the match operation.	c		e	f	g	a		i		j	k	l	m	n	b

		M_NONE	Specifies no internal conversion.	c		e	f	g	a		i		j	k	l	m	n	b

The following ContextId, DistTypeOrCallIntent, and ConversionModeOrComputeOption parameter settings specify the color calibration intent (quality) with which to perform the operation that Aurora Imaging Library uses to establish the mapping it associates to color-samples. These settings can increase or decrease the accuracy inherent in the operation. The ContextId parameter must indicate the identifier of a color matching context. Unless otherwise specified, values are available for all operations.

Except for converting RGB source color data to CIELAB or HSL when matching colors, the ConversionModeOrComputeOption parameter is unused and should be set to **M_DEFAULT** (same as M_NONE).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the color calibration intent (quality) of the mapping operation (relative color calibration)																		
ContextId			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	DistTypeOrCallIntent																	
		ConversionModeOrComputeOption																
Relative color calibration context ID			Specifies a relative color calibration context. If using M_GLOBAL_MEAN_VARIANCE, set this parameter to M_DEFAULT.															
	M_DEFAULT		Specifies the default color calibration intent. For M_COLOR_TO_COLOR and M_HISTOGRAM_BASED, the default is M_BALANCE. For M_GLOBAL_MEAN_VARIANCE, the default is M_GENERALIZATION.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_COMPUTE_ITEM_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPUTE_ITEM_PIXELS	Specifies that Aurora Imaging Library uses all pixel values (in the reference color-sample and the color-samples) to generate the color information that the operation requires.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPUTE_ITEM_STAT	Specifies that Aurora Imaging Library uses color statistics (in the reference color-sample and the color-samples) to generate the color information that the operation requires. You can only specify this value if using an M_COLOR_TO_COLOR operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_BALANCE	Specifies that the operation considers a moderate amount of color information to establish the mapping. Though this setting offers less accuracy than M_PRECISION, there is little risk of overfitting the color data. You cannot specify M_BALANCE if you are using an M_GLOBAL_MEAN_VARIANCE operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COMPUTE_ITEM_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPUTE_ITEM_PIXELS	Specifies that Aurora Imaging Library uses all pixel values (in the reference color-sample and the color-samples) to generate the color information that the operation requires.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPUTE_ITEM_STAT	Specifies that Aurora Imaging Library uses color statistics (in the reference color-sample and the color-samples) to generate the color information that the operation requires. You can only specify this value if using an M_COLOR_TO_COLOR operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERALIZATION	Specifies that the operation considers a minimal amount of color information to establish the mapping. This setting offers the least precise color mapping. However, there is little risk of overfitting the color data, and it is the fastest calculation, which can prove useful for large images.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_COMPUTE_ITEM_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPUTE_ITEM_PIXELS	Specifies that Aurora Imaging Library uses all pixel values (in the reference color-sample and the color-samples) to generate the color information that the operation requires.	c		e	f	g	a		i		j	k	l	m	n	b

		M_COMPUTE_ITEM_STAT	Specifies that Aurora Imaging Library uses color statistics (in the reference color-sample and the color-samples) to generate the color information that the operation requires. You can only specify this value if using an M_COLOR_TO_COLOR operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PRECISION		Specifies that the operation considers a high amount of color information to establish the mapping. This setting offers the most precise color mapping between the reference color-sample and the color-samples. However, it can potentially overfit the color data. For example, if you perform the relative color calibration on grabbed images containing outlier colors that were not considered during preprocessing, incorrect color transformations can occur. You cannot specify M_PRECISION if you are using an M_GLOBAL_MEAN_VARIANCE operation.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_COMPUTE_ITEM_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPUTE_ITEM_PIXELS	Specifies that Aurora Imaging Library uses all pixel values (in the reference color-sample and the color-samples) to generate the color information that the operation requires.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPUTE_ITEM_STAT	Specifies that Aurora Imaging Library uses color statistics (in the reference color-sample and the color-samples) to generate the color information that the operation requires. You can only specify this value if using an M_COLOR_TO_COLOR operation.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolStream

Synopsis

Load, restore, or save an Aurora Imaging Library color matching or relative color calibration context from/to a file or memory stream.

Syntax

```
void McolStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextColIdPtr,       //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save an Aurora Imaging Library color matching or relative color calibration context from/to a file or memory stream.

To inquire the number of bytes necessary to save the context, you should first call this function (**McolStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. This function is equivalent to a file saved using McolSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file. All information about the previously allocated context (and the color-samples therein) is saved. However, preprocessing information is not saved.

All of the context's settings (and color-samples) that were in effect when the context was saved will be restored. A loaded or restored context is not preprocessed; therefore, you must call McolPreprocess() before performing McolMatch() (color matching) or McolTransform() (relative color calibration).

Using **McolStream**(), you can choose to save a backwards-compatible version of the context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Settings that do not exist in the lower version will be filled with default values when the context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the context. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the type of stream																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextColIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the context, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> The size of the context varies depending on the Aurora Imaging Library version that you specify.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextColIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
ContextColIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a color matching or relative color calibration context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the color matching or relative color calibration context must be compatible when saved. The size of a color matching or relative color calibration context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b

M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextColIdPtr	Specifies the address of the variable containing the identifier of the color matching or relative color calibration context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a previously allocated color matching or relative color calibration context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Color matching contexts typically have an MCOL file extension. Relative color calibration context typically have an MCCC file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextColIdPtr	Specifies the address of the variable containing the identifier of the color matching or relative color calibration context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Restores a color matching or relative color calibration context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Color matching contexts typically have an MCOL file extension. Relative color calibration context typically have an MCCC file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b

SysId	Specifies the system on which to allocate the color matching or relative color calibration context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextColIdPtr	Specifies the address of the variable in which to write the identifier of the color matching or relative color calibration context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves a color matching or relative color calibration context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Aurora Imaging software products, when saving a color matching context to a file, use the MCOL file extension. When saving a relative color calibration context to a file, use the MCCC file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		Specifies the Aurora Imaging Library version with which the color matching or relative color calibration context must be compatible when saved. Note that the color matching or relative color calibration context can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextColIdPtr		Specifies the address of the variable containing the identifier of the color matching or relative color calibration context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.

McolTransform

Synopsis

Transform color data using relative color calibration.

Syntax

```
void McolTransform(
    AIL_ID ContextId,           //in
    AIL_INT SampleIndexOrLabel, //in
    AIL_ID SrcImageBufId,       //in
    AIL_ID DestImageBufId,      //out
    AIL_INT64 ControlFlag       //in
)
```

Description

This function transforms a source image's color data according to the mapping associated with a color-sample in a relative color calibration context. The transformed image is written in the destination image buffer.

This function supports in-place processing; you can specify the identifier of the same image buffer for both the source and the destination.

The mapping associated with the color-sample (SampleIndexOrLabel) is established with McolDefine() and McolPreprocess(). McolDefine() defines the color-samples, and reference color-sample. McolPreprocess() establishes the mapping with which to interpret the color data of each color-sample so it is similar to the color data of the reference color-sample (for example, similar color temperature and distribution). By applying the color-sample mapping, **McolTransform**() corrects the color data of its source images to resemble the color data of the reference color-sample. You can then use these transformed source images, which now resemble each other, to perform other color operations, such as color matching (McolMatch()).

To influence the transformation, you can alter how Aurora Imaging Library establishes the color-sample mapping by changing the processing strategy with McolSetMethod(). Such changes require you to call McolPreprocess() before **McolTransform**().

Parameters

ContextId

Specifies the identifier of the relative color calibration context containing the color-sample and its associated mapping. The relative color calibration context must have been previously allocated on the required system using McolAlloc() with M_COLOR_CALIBRATION_RELATIVE. The relative color calibration context must also have been previously preprocessed using McolPreprocess().

Aurora Imaging Library assumes all color data you use with a relative color calibration context is 8-bit RGB (3-band). If color data is inconsistent (for example, between the source image and the color-sample), Aurora Imaging Library still processes it all as 8-bit RGB, which can produce misleading results.

SampleIndexOrLabel

Specifies the index or label of a color-sample in the specified relative color calibration context. This function transforms the color of the source image according to the mapping associated to the color-sample. Ensure the color-sample is appropriate for the specified source image.

For specifying a color-sample																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SAMPLE_INDEX(AIL_INT SampleIndex)	Specifies the index of a color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleIndex Specifies an index value greater than or equal to 0.															
M_SAMPLE_LABEL(AIL_INT SampleLabel)	Specifies the label of a color-sample.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SampleLabel Specifies a label value greater than 0.															

SrcImageBufId

Specifies the identifier of the source image with which to perform the relative color calibration. The buffer must be an 8-bit color (3-band) unsigned image buffer (same as the destination).

DestImageBufId

Specifies the identifier of destination image in which to write the transformed color. The buffer must be an 8-bit color (3-band) unsigned image buffer (same as the source).

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailcolor11.lib.
DLL	Requires ail11.dll; ailcolor11.dll.